

Sequence Modeling Architectures I

From RNN to Transformer

Chuyang Zhou

With slides from A/Prof. Chang Xu
School of Computer Science



THE UNIVERSITY OF
SYDNEY



Quick Review

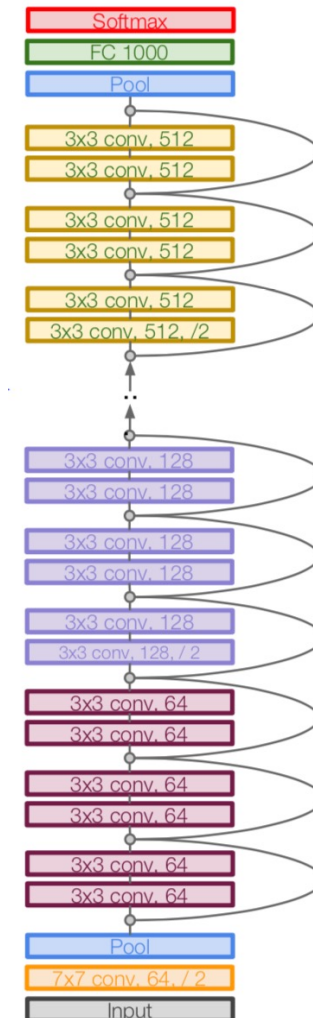
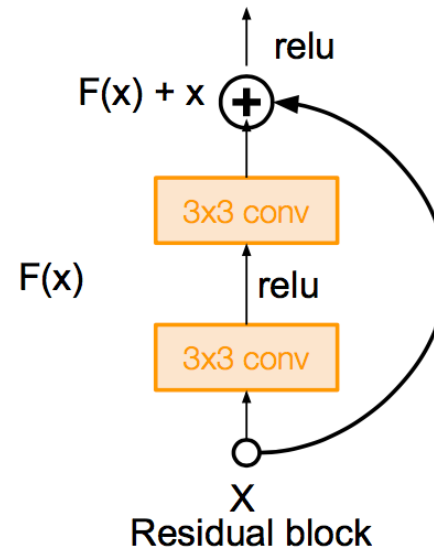
CNN Architecture: Part I

□ ResNet [He et al., 2015]

Very deep networks using residual connections

Basic design:

- all 3x3 conv (almost)
- spatial size /2 => #filter x2
- just deep
- average pooling (remove FC)
- no dropout



ResNet

Outline

- Recurrent Neural Network
- Long Short-Term Memory
- Gated Recurrent Unit
- Applications
- Transformer
 - Self-Attention
 - Multi-head Self-Attention

NLP Example Application



关注

There's a new star in town. And it's going to change the way you experience everything. Introducing the #GalaxyS9. Find out more.

翻译自英语



上午1:05 · 2018年2月26日



Samsung Mobile US @SamsungMobileUS · 2月25日

Nothing wrong with sticking to what you know.

翻译自英语

4 4 19



Emma Driver @emmalb1988 · 2月25日

Absolutely. Have nothing **but** love for my Samsung. Spent 7 years as an iPhone user **but** now I'd never look back.

翻译自英语

7



Shubham Kumar @iamshubhmKr · 2月25日

回复 @SamsungMobile

Same old boring Samsung .

翻译自英语

1 1 5



Image credit to <https://twitter.com/SamsungMobile/status/967807667463958531>

Recurrent Neural Network

Recurrent Neural Network

- Input

- Word sequence

- Each word is represented by vector

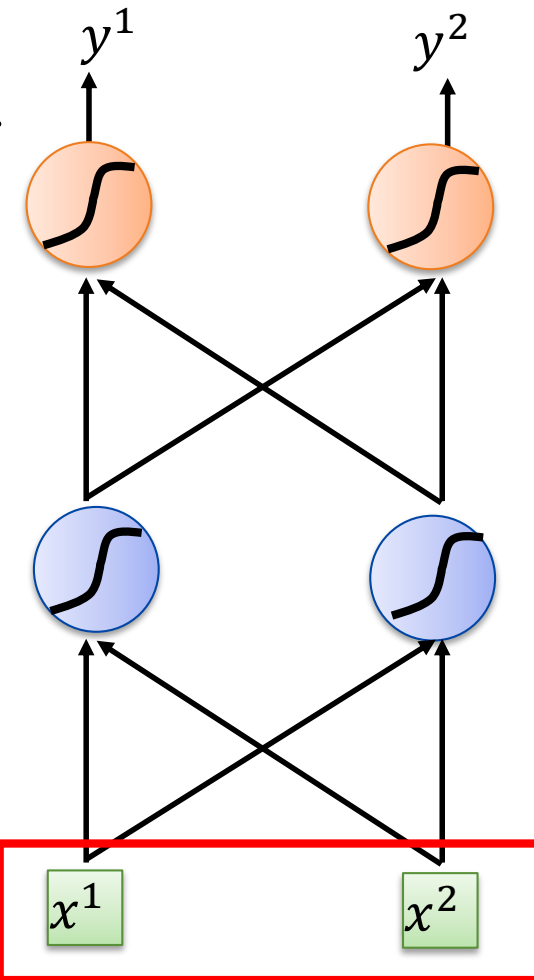
- Word Representation Methods

- One hot encoding

- Word hash

- Word embedding

“Complicated”



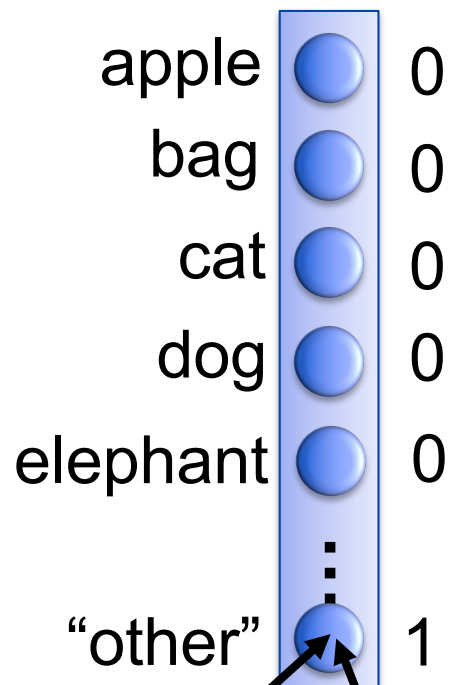
Recurrent Neural Network

- ❑ One hot encoding
 - ❑ The length of vector is lexicon size
 - ❑ Each dimension corresponds to a word in the lexicon
 - ❑ The dimension for the word is 1, and others are 0
- lexicon = {apple, bag, cat, dog, elephant}

Word	D_1	D_2	D_3	D_4	D_5
Apple	1	0	0	0	0
Bag	0	1	0	0	0
Cat	0	0	1	0	0
Dog	0	0	0	1	0
Elephant	0	0	0	0	1

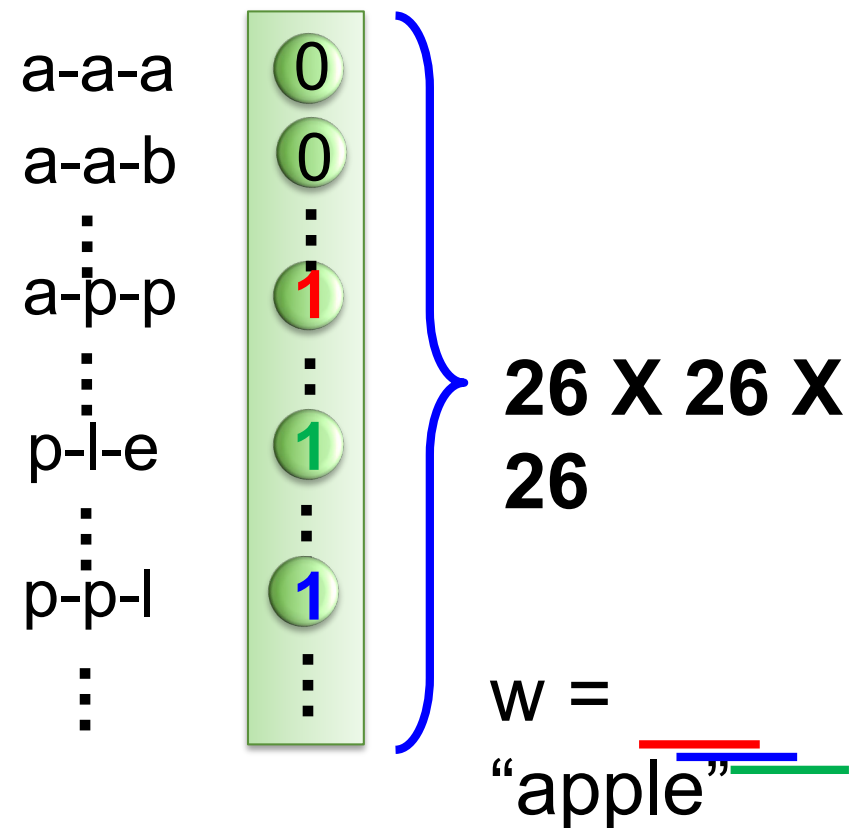
Recurrent Neural Network

Dimension for “Other”



$w = \text{“Gandalf”}$ $w = \text{“Sauron”}$

Word hashing



Recurrent Neural Network

□ Output

- Probability distribution that the sentence belongs to negative or positive.

Complicated? Yes,
and a little slow, too.



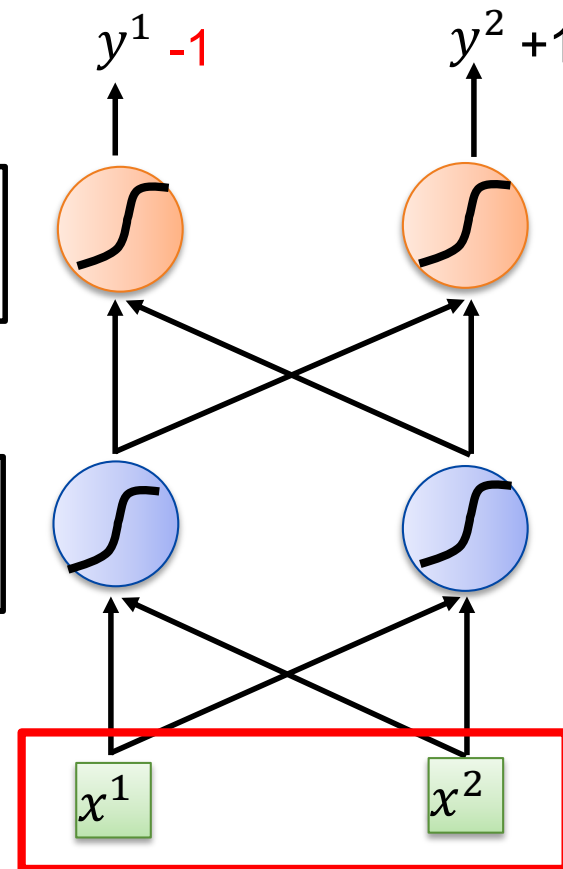
$$\begin{matrix} y^1 = 0.9 \\ y^2 = 0.1 \end{matrix}$$

But the animation is as
colorful as the story and
emotions do eventually reach
the desired level.

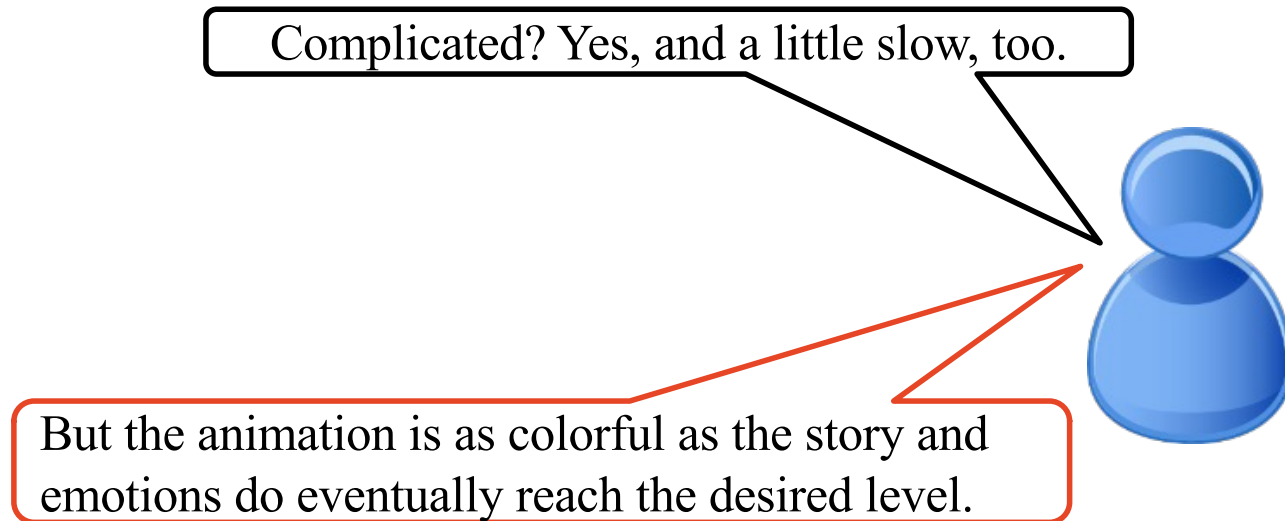


$$\begin{matrix} y^1 = 0.3 \\ y^2 = 0.7 \end{matrix}$$

Complicated? Yes,
and a little slow, too.



Recurrent Neural Network



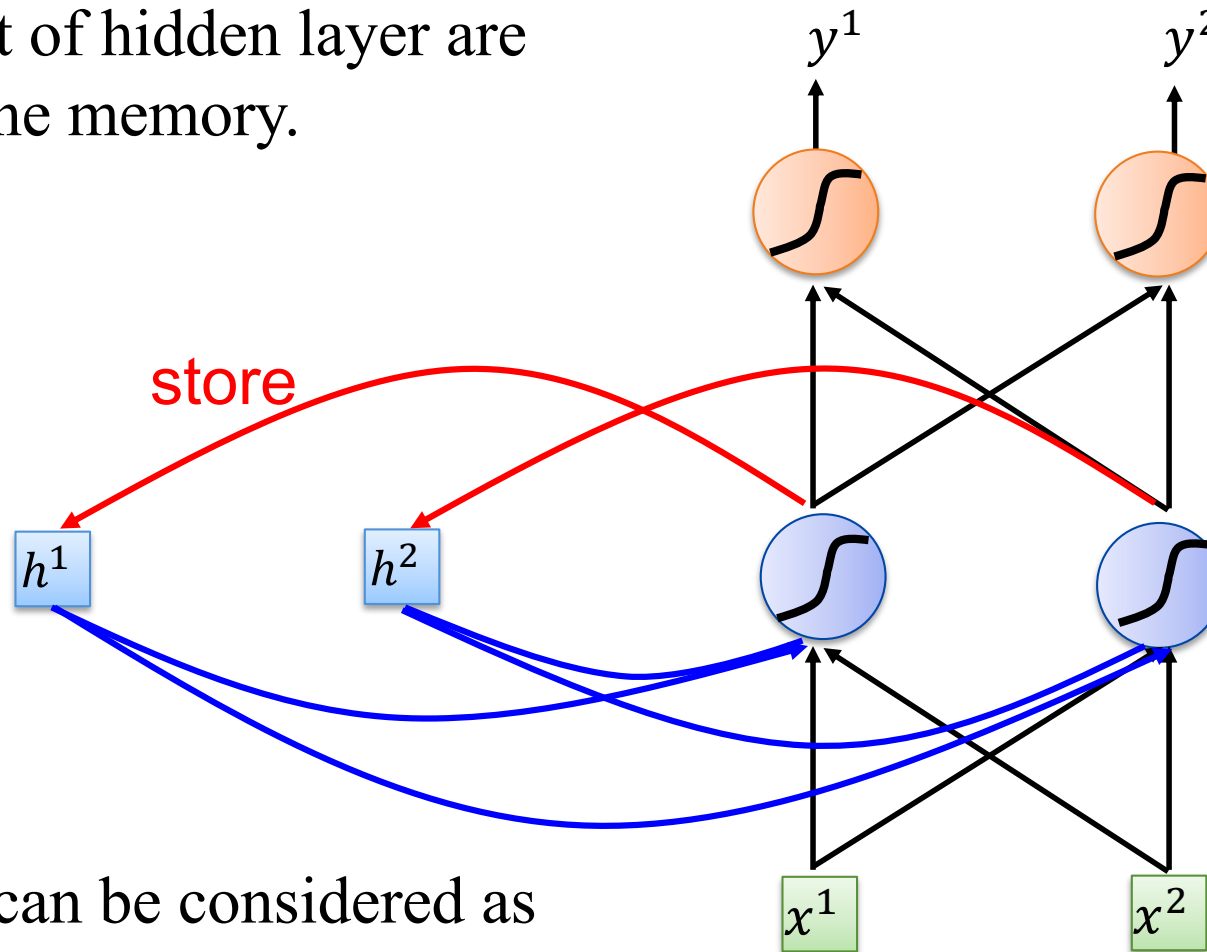
Negative OR Positive



Need
Memory

Recurrent Neural Network

The output of hidden layer are stored in the memory.

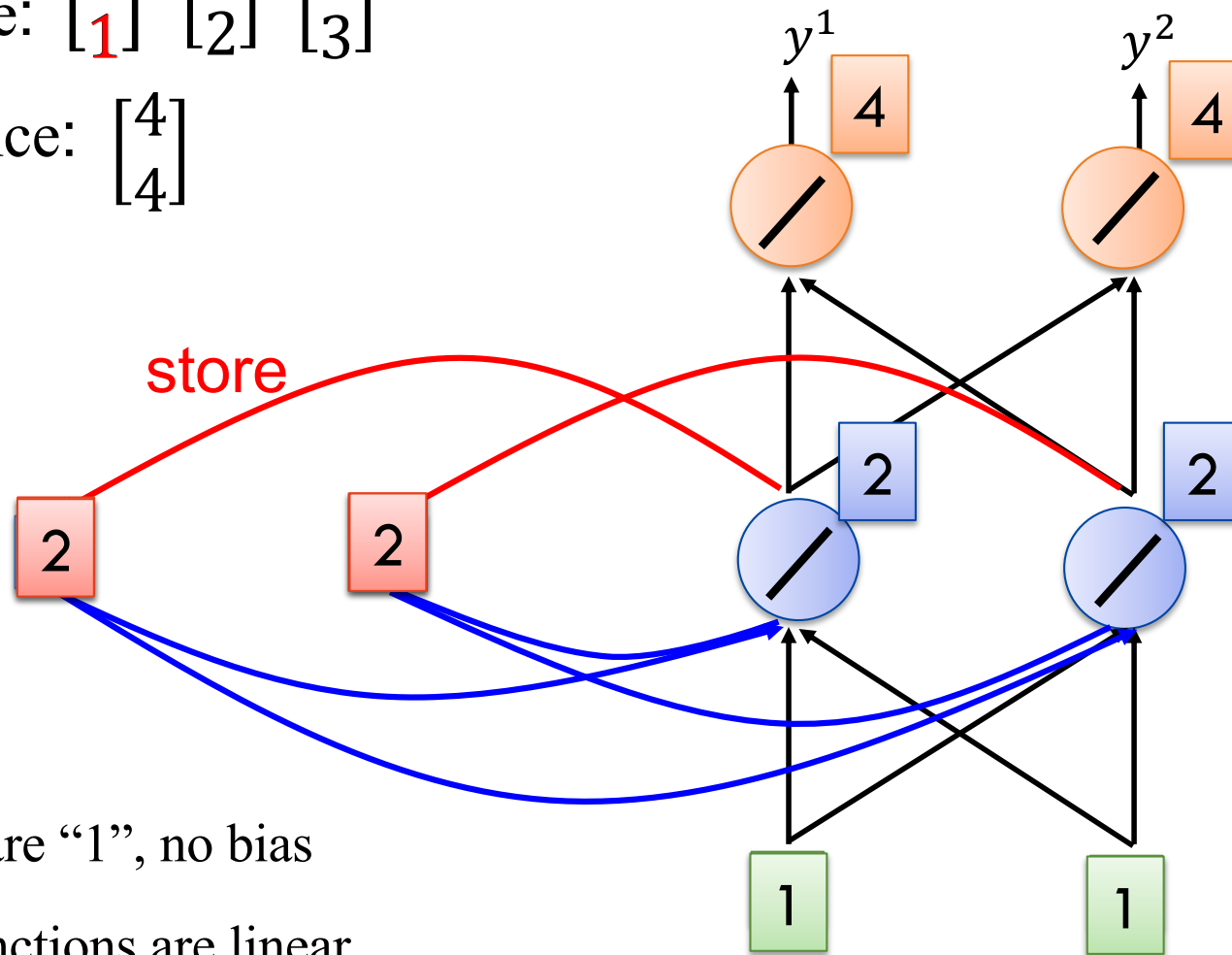


Memory can be considered as another input.

Recurrent Neural Network

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$ $\begin{bmatrix} 3 \\ 3 \end{bmatrix}$

Output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$



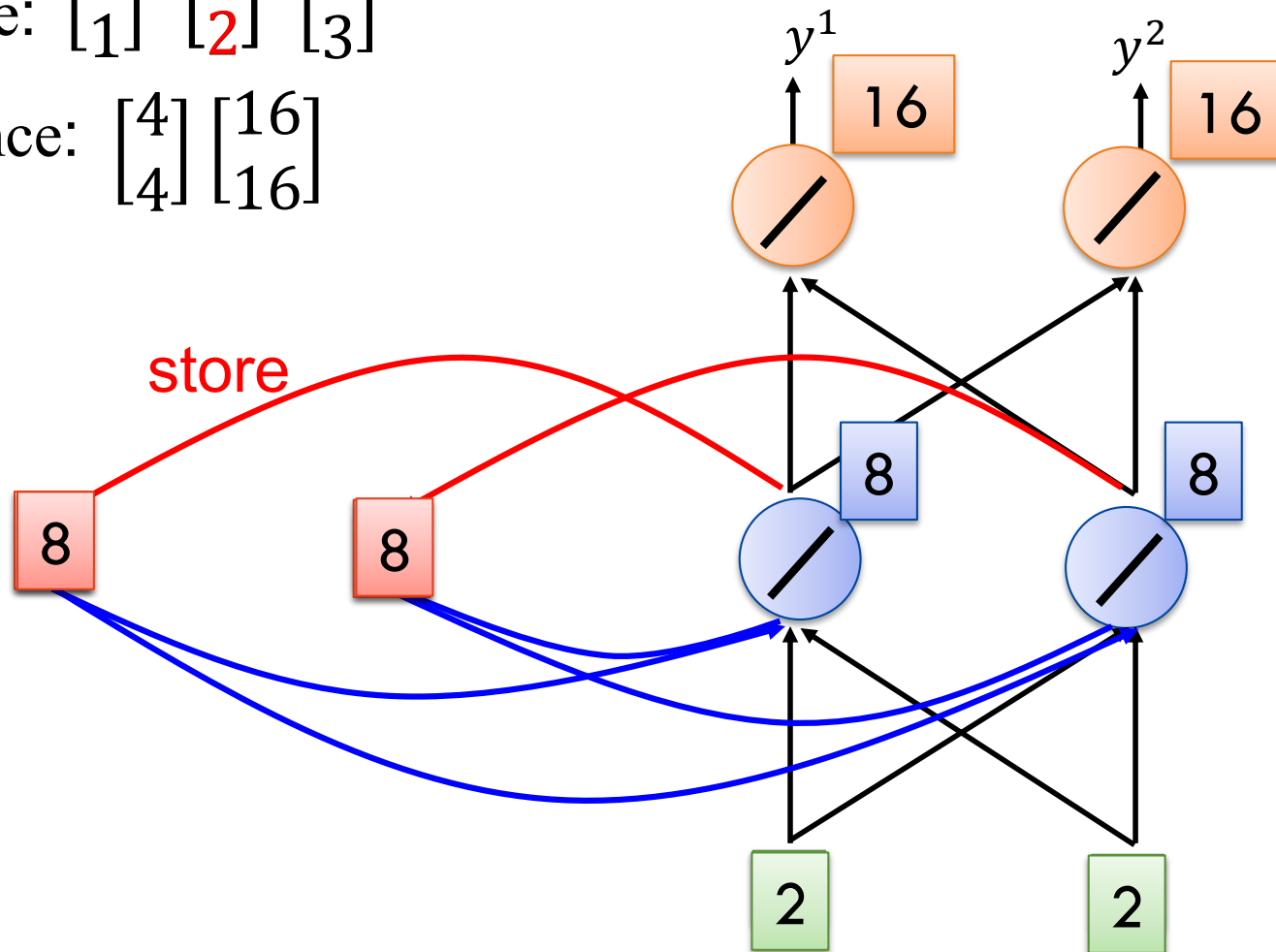
All the weights are “1”, no bias

All activation functions are linear

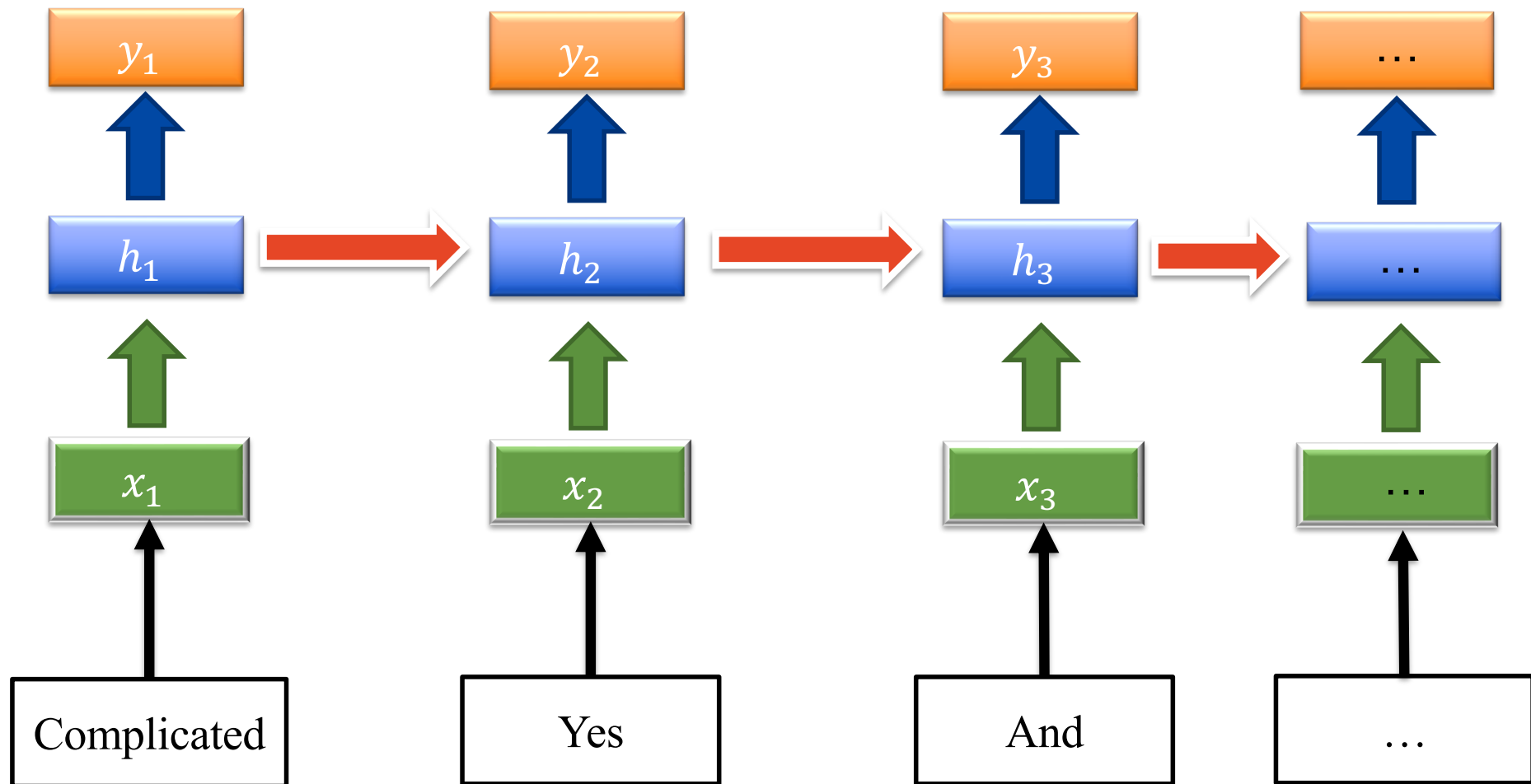
Recurrent Neural Network

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$ $\begin{bmatrix} 3 \\ 3 \end{bmatrix}$

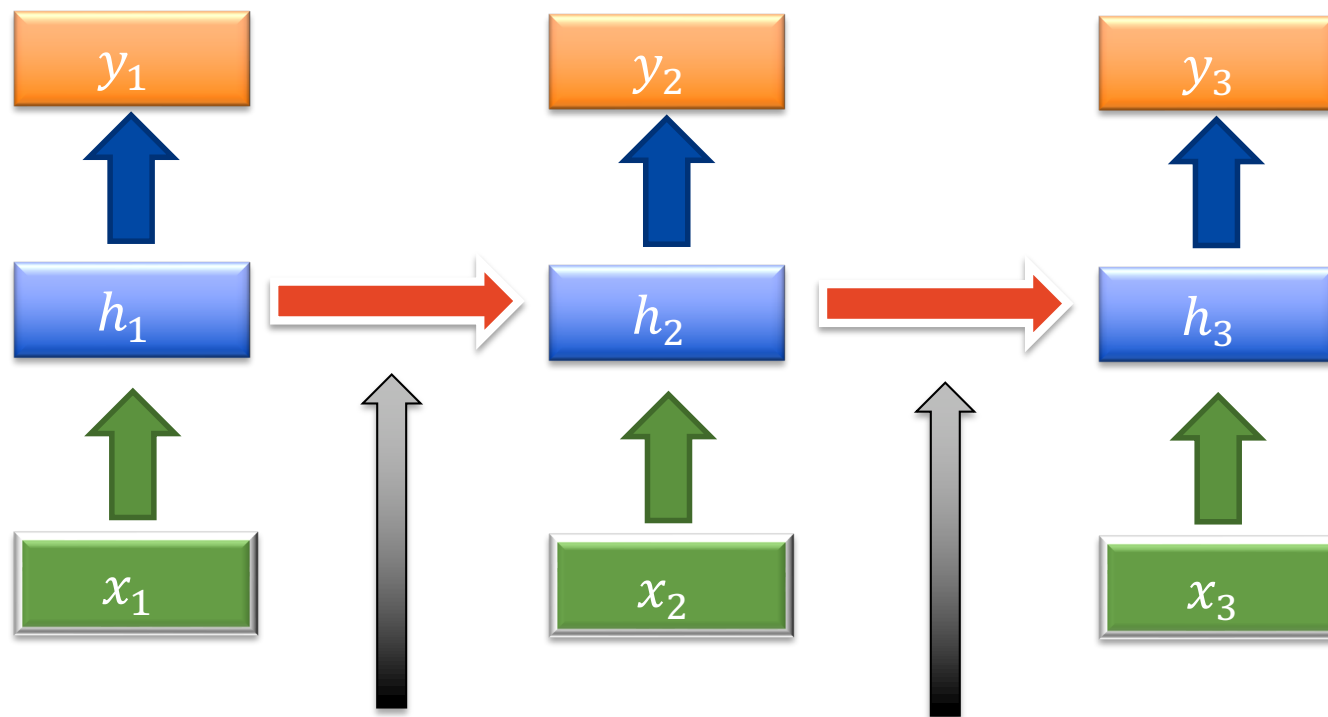
Output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$ $\begin{bmatrix} 16 \\ 16 \end{bmatrix}$



Recurrent Neural Network



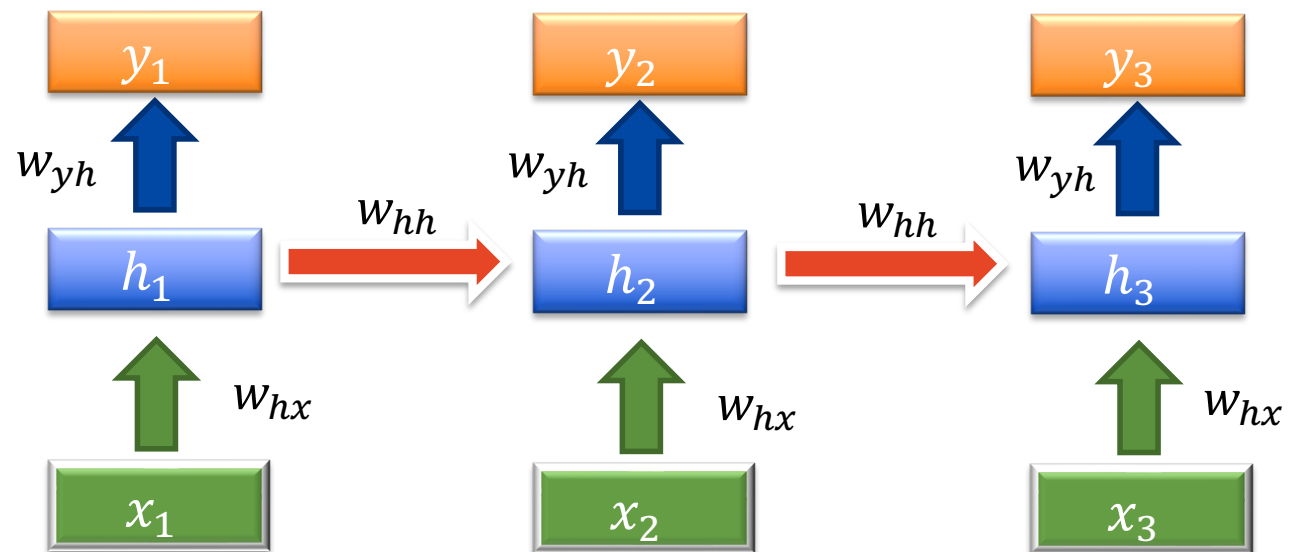
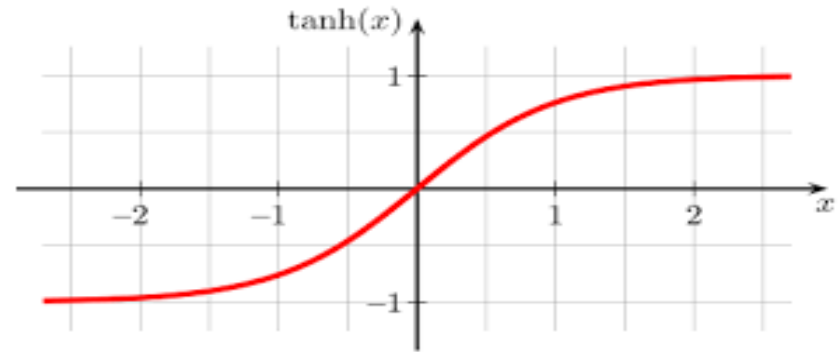
Recurrent Neural Network



Memory can be considered as another input.

Recurrent Neural Network

- x_t is the input
- h_t is the hidden state
- y_t is the output
- $h_0 = \vec{0}$
- $h_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$
- $y_t = w_{yh}h_t$



Recurrent Neural Network

□ Vanishing gradient problem

$$\square h_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$$

$$\square y_t = w_{yh}h_t$$

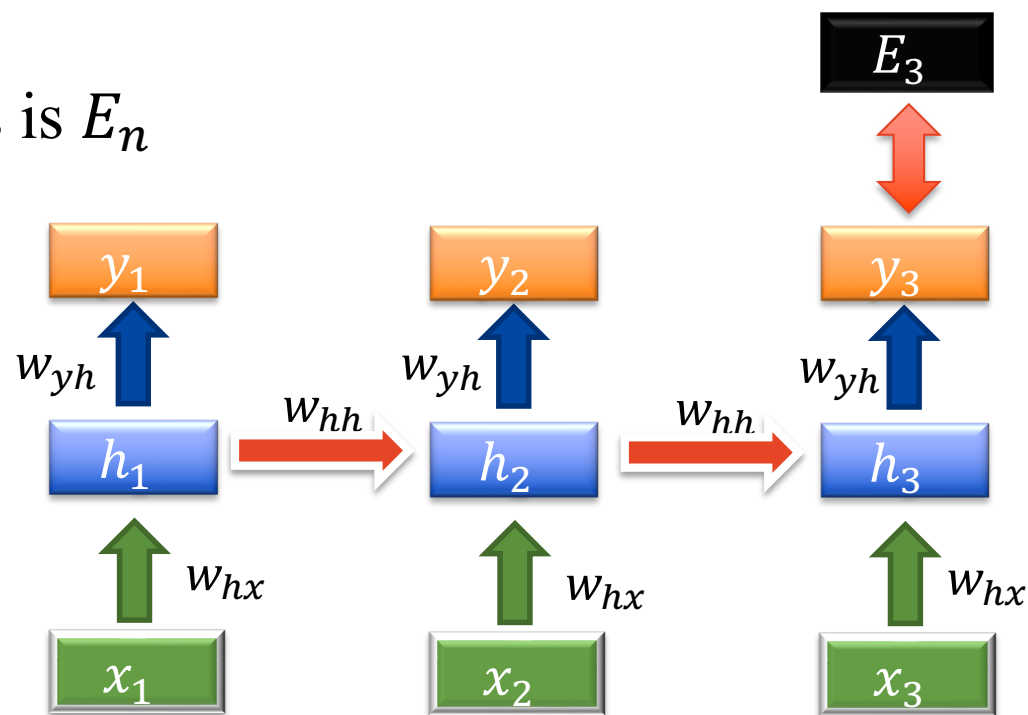
□ For every step, its loss is E_n

$$\frac{\partial E_3}{\partial w_{hh}} = \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial w_{hh}}$$

$$h_3 = \tanh(w_{hh}h_2 + w_{hx}x_3 + b_h)$$

$$\frac{\partial h_3}{\partial w_{hh}} = (\tanh)' \left(h_2 + w_{hh} \frac{\partial h_2}{\partial w_{hh}} \right)$$

$$\frac{\partial h_2}{\partial w_{hh}} = (\tanh)' \left(h_1 + w_{hh} \frac{\partial h_1}{\partial w_{hh}} \right)$$



$$(\tanh)'w_{hh} (\tanh)'w_{hh} (\tanh)'w_{hh} \dots$$

Recurrent Neural Network

□ Vanishing gradient problem

□ $h_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$

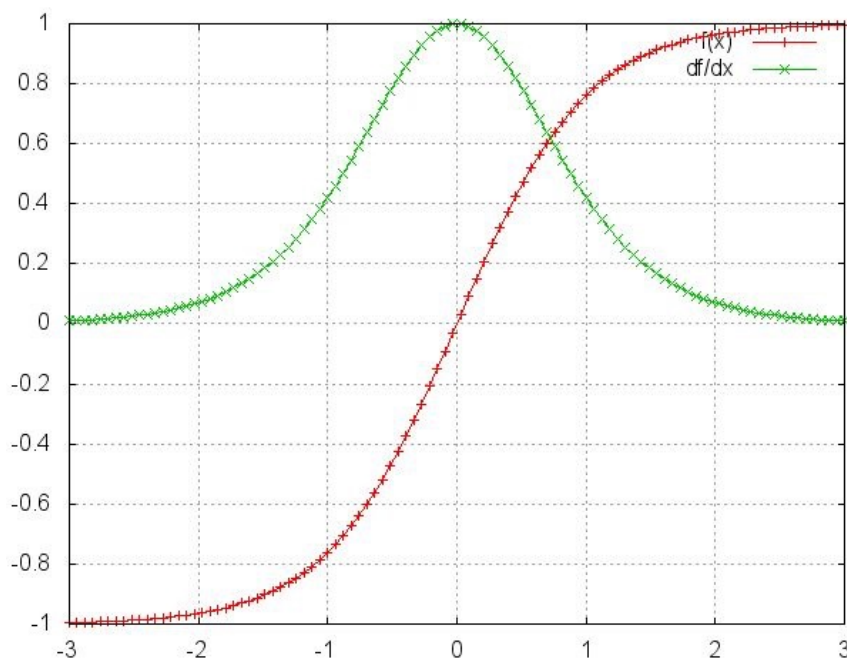
□ $y_t = w_{yh}h_t$

□ For every step, its loss is E_n

$$\begin{aligned} 0.9^{1000} &\approx 0 \\ 1.01^{1000} &\approx 21000 \end{aligned}$$

$$\frac{\partial E_3}{\partial w_{hh}} = \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial w_{hh}}$$

$$(\tanh)'w_{hh}(\tanh)'w_{hh}(\tanh)'w_{hh} \cdots$$



$$(\tanh)' \leq 1$$

If $0 < w_{hh} < 1$, **Vanishing Gradients**

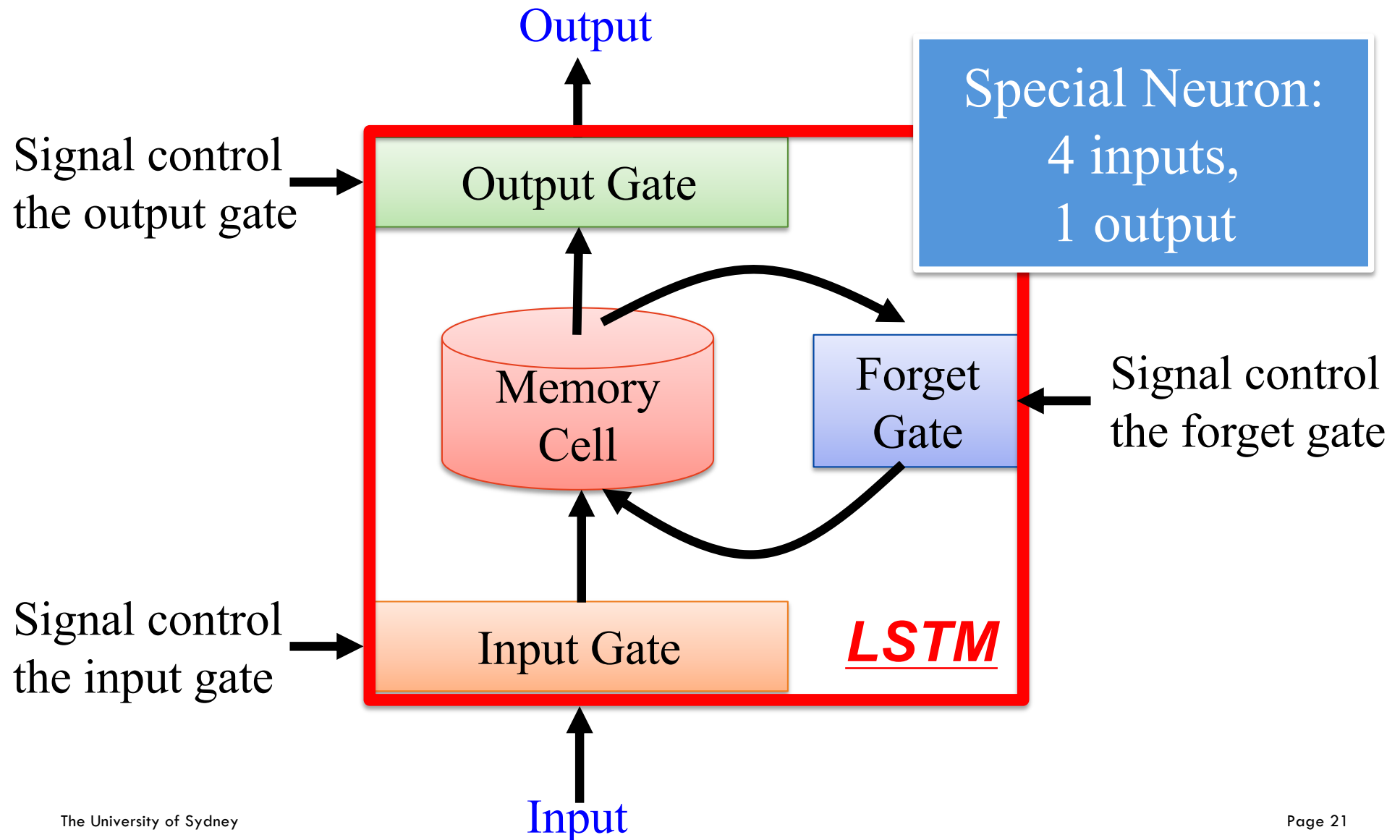
$$(\tanh)'w_{hh}(\tanh)'w_{hh}(\tanh)'w_{hh} \cdots \rightarrow 0$$

If w_{hh} is larger, **Exploding Gradients**

$$(\tanh)'w_{hh}(\tanh)'w_{hh}(\tanh)'w_{hh} \cdots \rightarrow \infty$$

Long Short-Term Memory

Long Short-Term Memory



Long Short-Term Memory

Inputs

x_t : Current Input

h_{t-1} : Previous hidden state

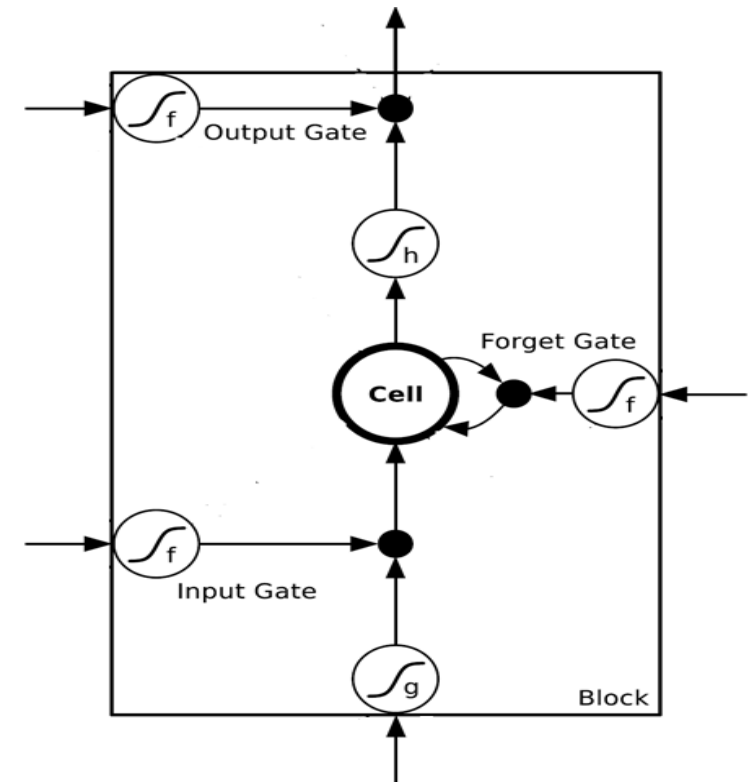
Memory

c_{t-1} : Previous cell state

Outputs

h_t : Updated hidden state

c_t : Updated cell state



Long Short-Term Memory

❑ Forget gate

$$- f_t = \sigma(w_{fh}h_{t-1} + w_{fx}x_t + b_f)$$

❑ Input gate

$$- i_t = \sigma(w_{ih}h_{t-1} + w_{ix}x_t + b_i)$$

❑ Output gate

$$- o_t = \sigma(w_{oh}h_{t-1} + w_{ox}x_t + b_o)$$

❑ Candidate cell state

$$- \tilde{c}_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$$

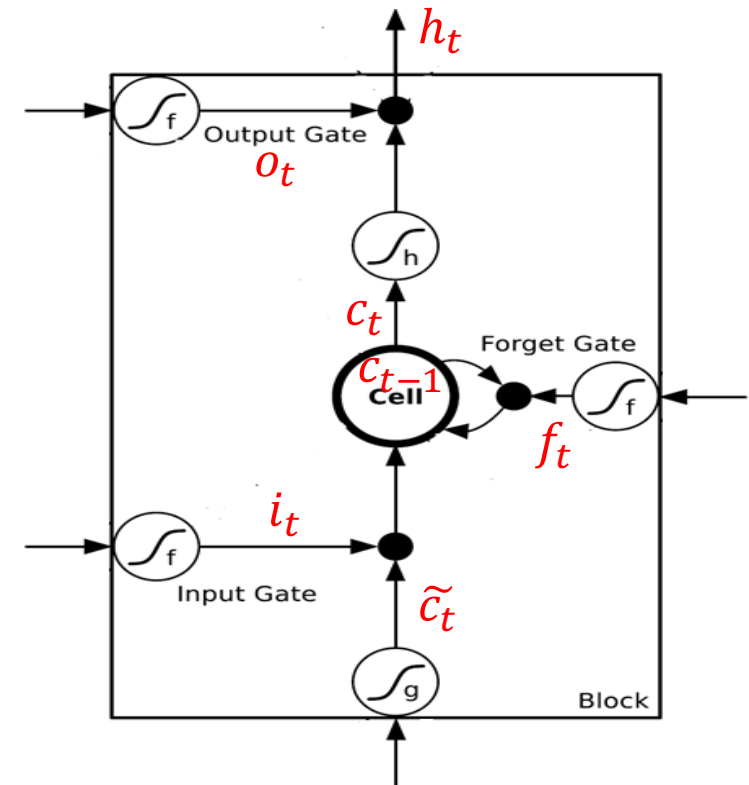
❑ Update cell state

$$- c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$$

❑ Hidden state

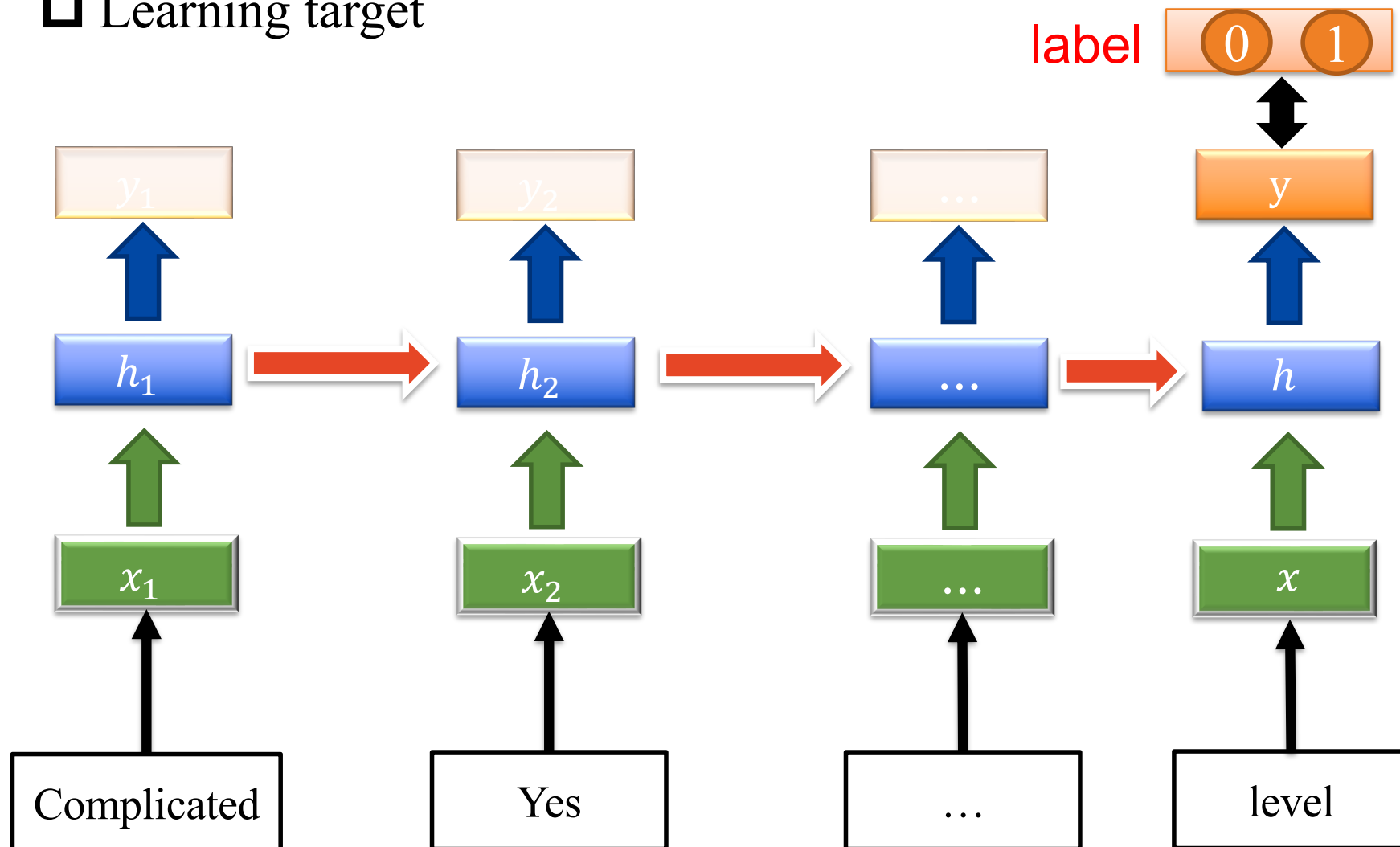
$$- h_t = o_t * \tanh(c_t)$$

∗: Element-wise multiplication



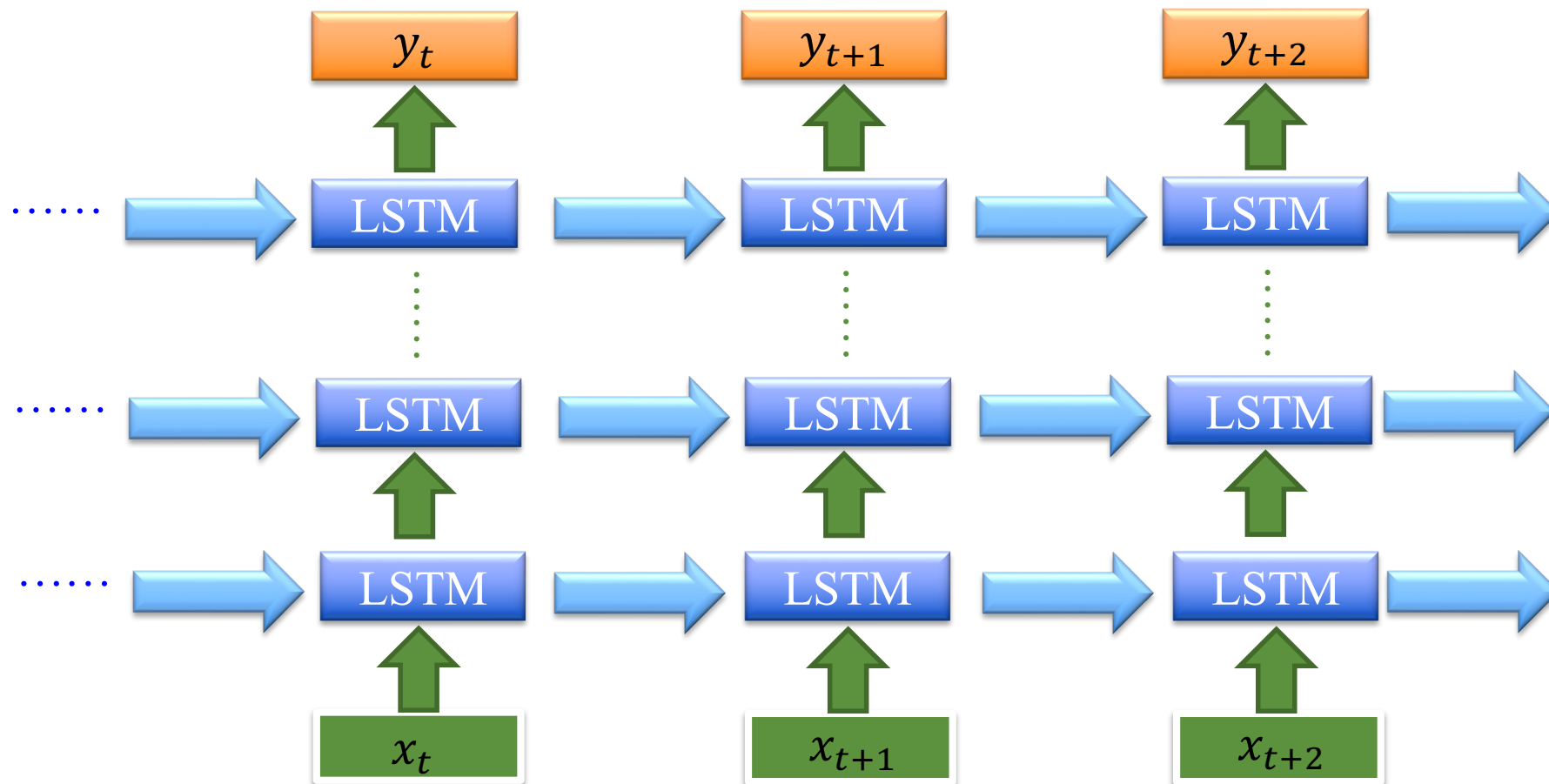
Long Short-Term Memory

□ Learning target



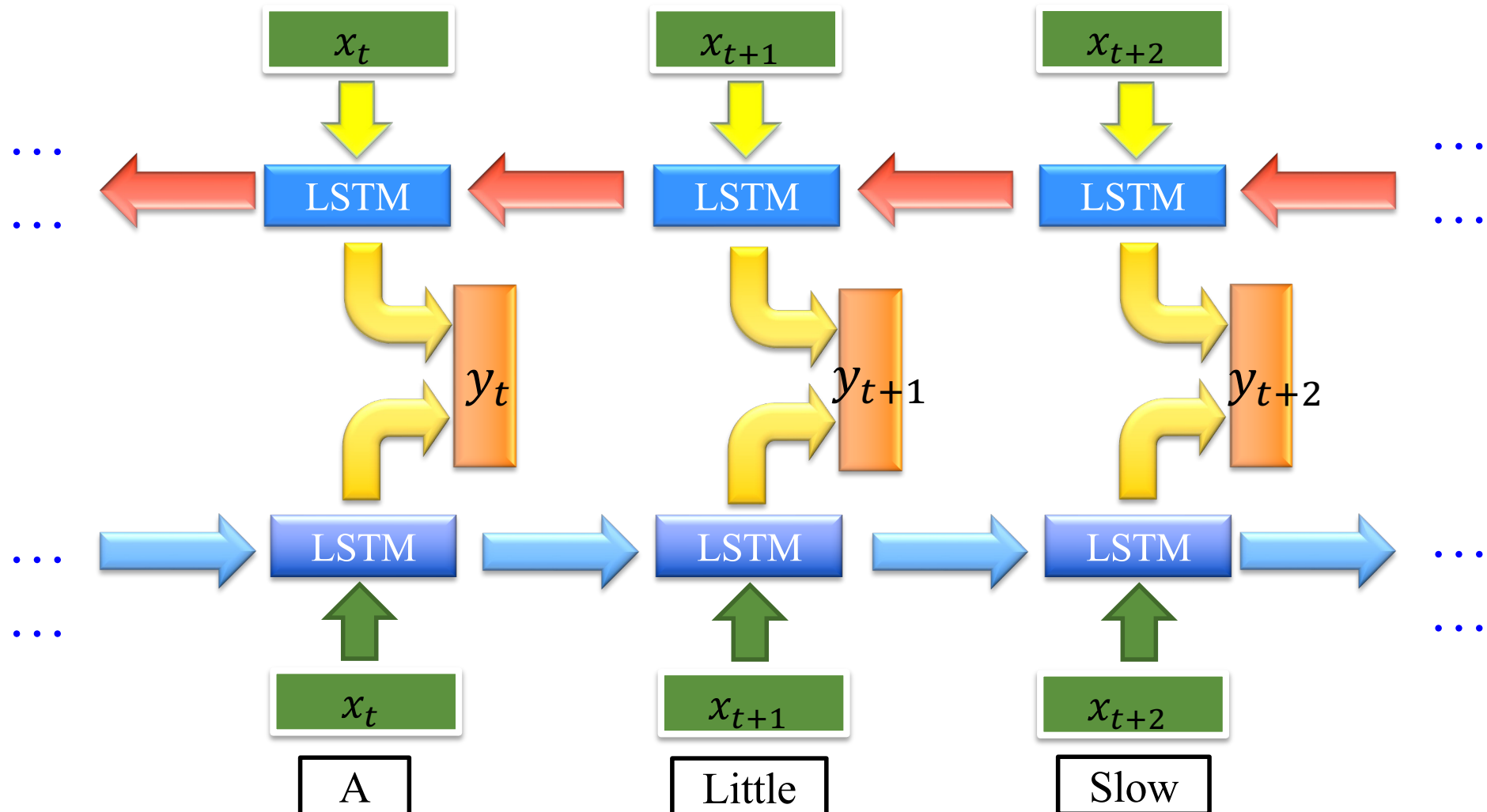
Long Short-Term Memory

□ Stacked LSTM

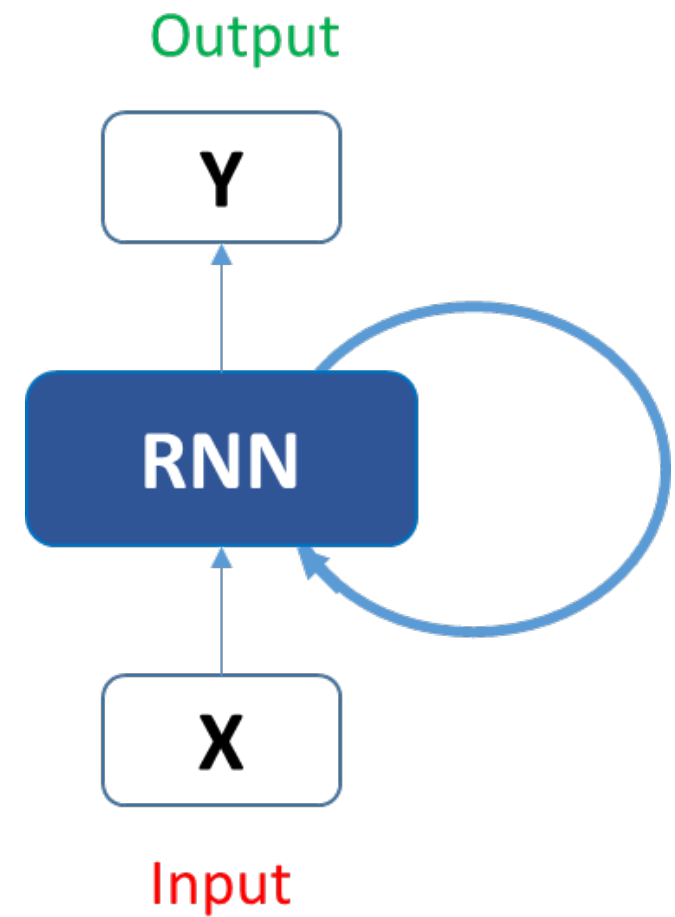
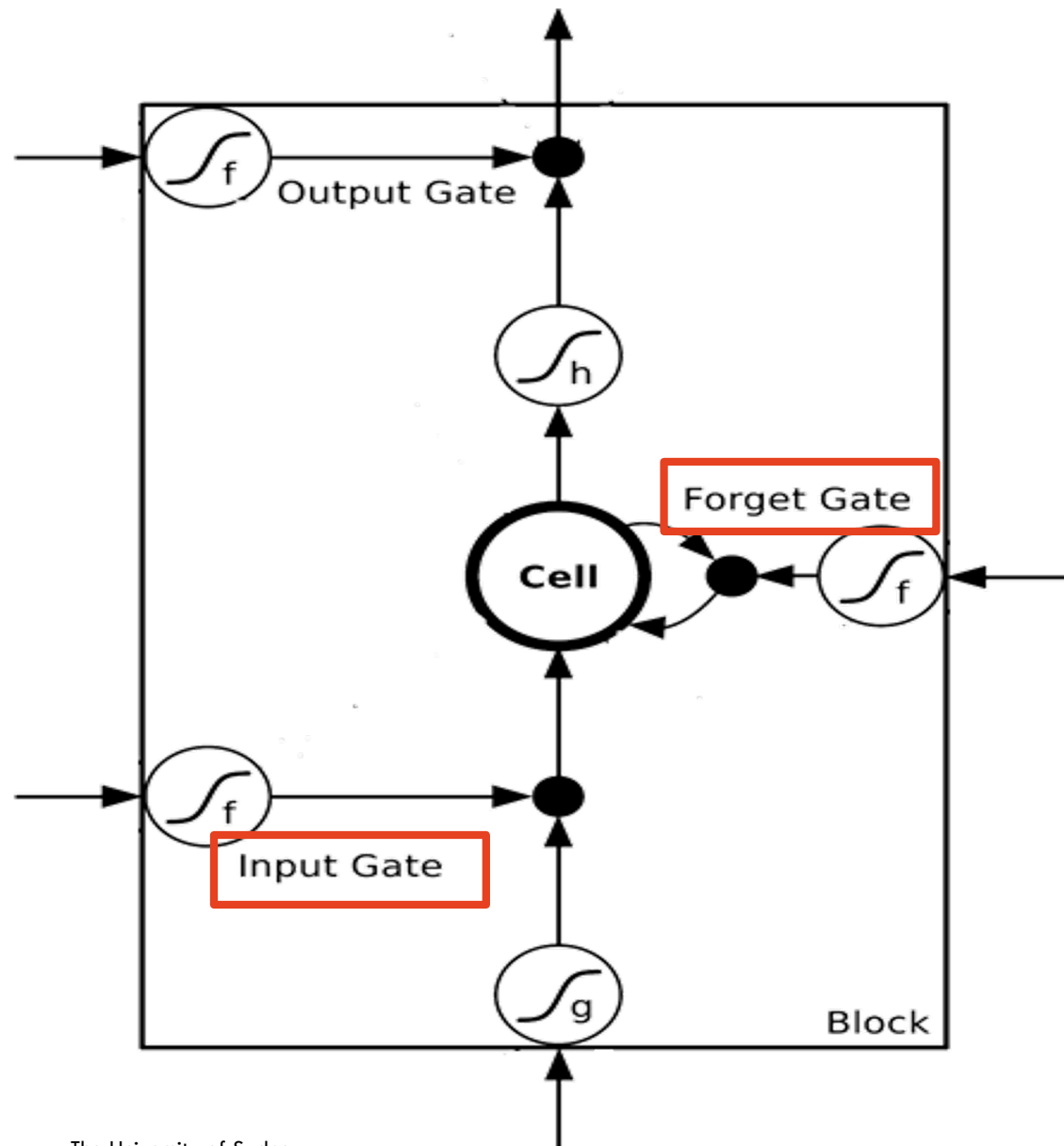


Long Short-Term Memory

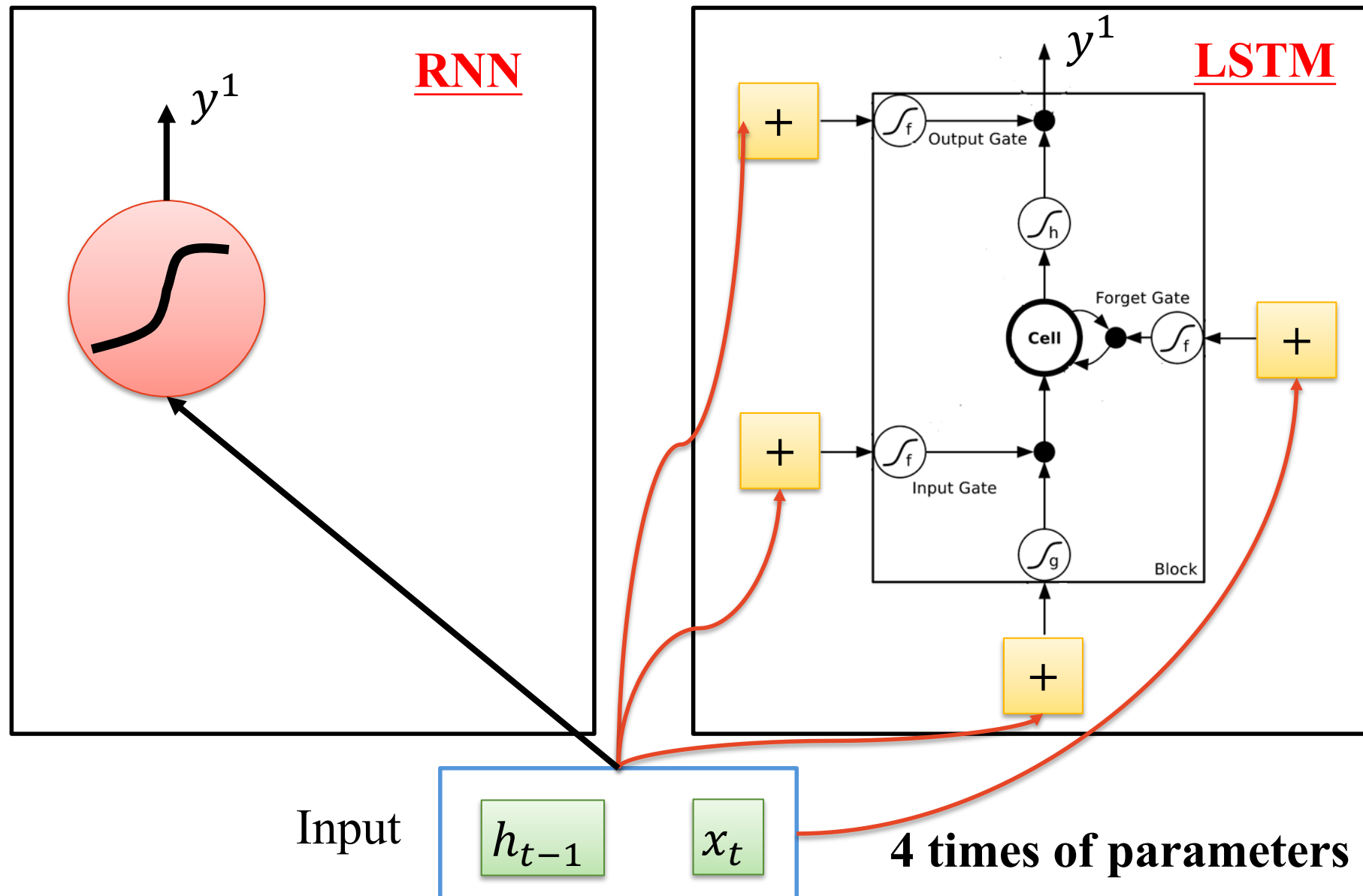
□ Bidirectional RNN



LSTM V.S. RNN -- Memory



LSTM vs RNN -- Parameters



LSTM vs RNN – Vanishing gradient

□ RNN

$$\bullet h_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$$

□ LSTM

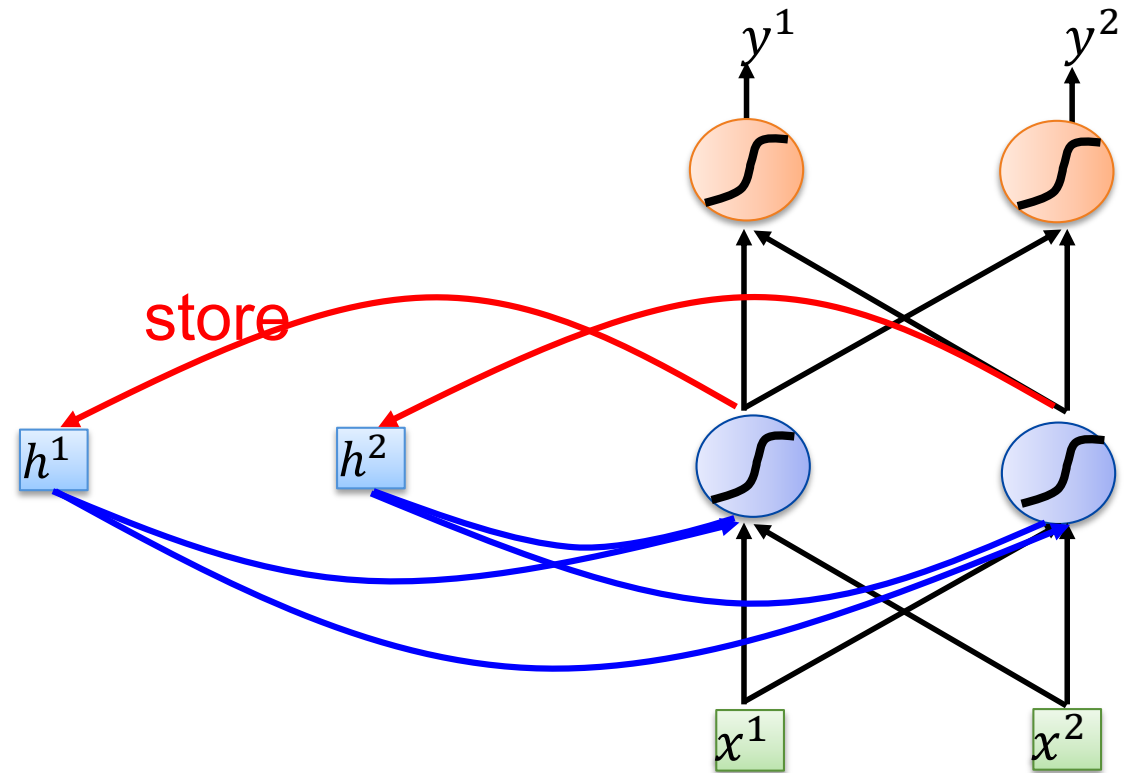
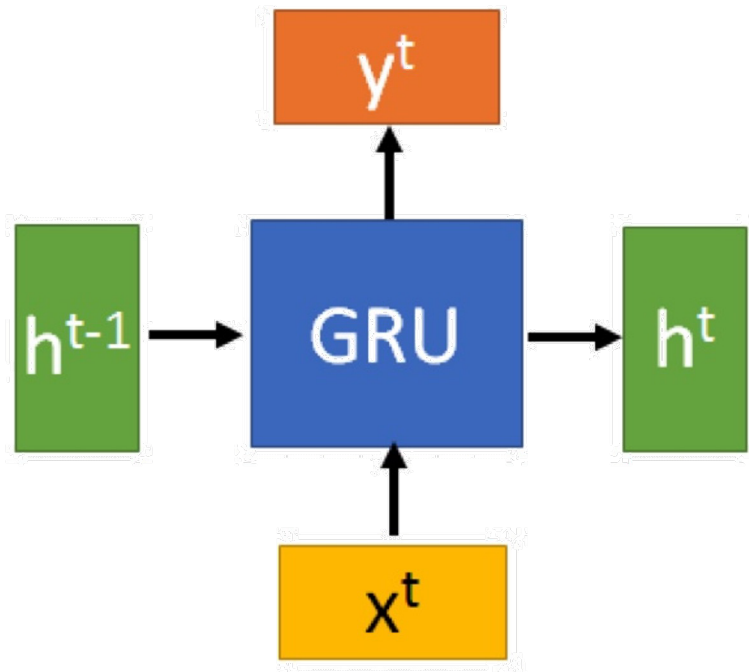
$$\bullet c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$$

$$\frac{\partial c_t}{\partial c_{t-1}} = \frac{\partial f_t}{\partial c_{t-1}} c_{t-1} + f_t + \frac{\partial i_t}{\partial c_{t-1}} \tilde{c}_t + \frac{\partial \tilde{c}_t}{\partial c_{t-1}} i_t$$

LSTMs solve the problem using a unique additive gradient structure that includes direct access to the forget gate's activations, enabling the network to encourage desired behaviour from the error gradient using frequent gates update on every time step of the learning process.

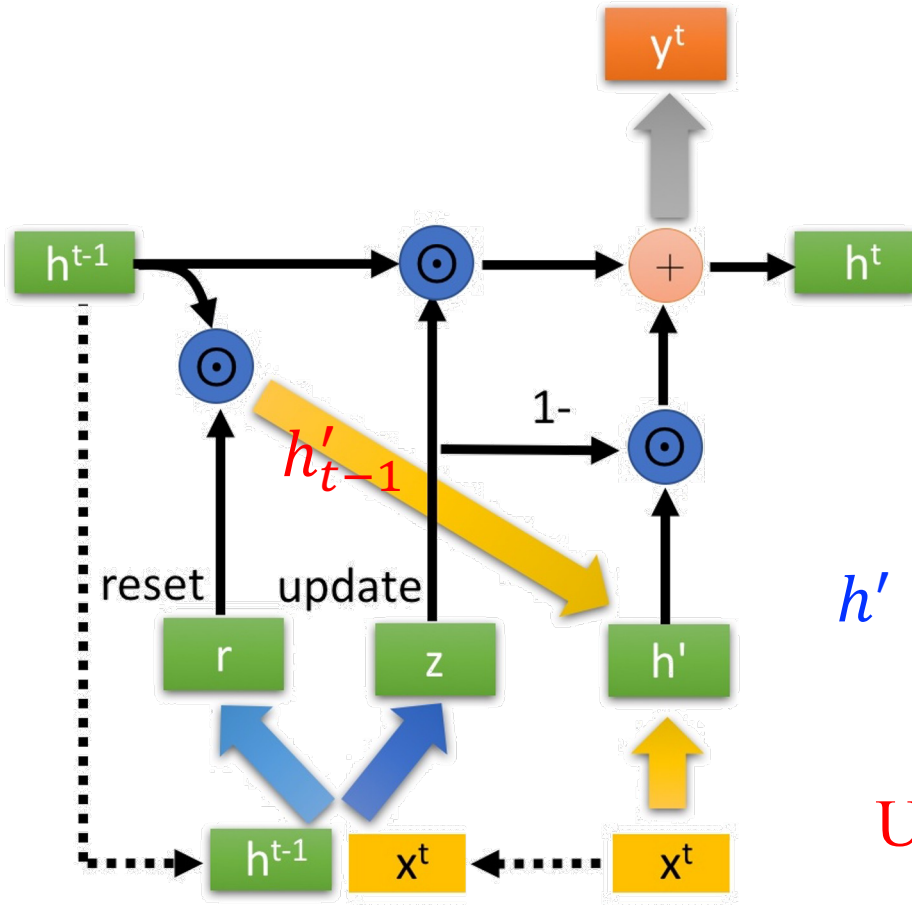
Gate Recurrent Unit

Gate Recurrent Unit



Vanilla RNN

Gate Recurrent Unit



Reset Gate

$$r_t = \sigma(w_{rh}h_{t-1} + w_{rx}x_t + b_r)$$

$$h'_{t-1} = r_t * h_{t-1}$$

$$h' = \tanh(w_{hh}h'_{t-1} + w_{hx}x_t + b_h)$$

Update Gate (forget gate + input gate)

$$z_t = \sigma(w_{zh}h_{t-1} + w_{zx}x_t + b_z)$$

$$h_t = z_t * h_{t-1} + (1 - z_t) * h'$$

GRU vs LSTM

GRU

- **Single hidden state** h_t
- **Two gates:**
 - Reset gate
 - controls how much past information is used to form the candidate state
 - Update gate
 - controls how much old state is retained versus replaced

LSTM

- Separate **cell state** c_t and **hidden state** h_t
- **Three gates:**
 - Forget gate
 - controls how much old memory to keep
 - Input gate
 - controls how much new information to write
 - Output gate
 - controls how much memory to expose as hidden state

RNN Applications

Application

- ❑ Visual Question Answer (VQA)
- ❑ Reading Comprehension
- ❑ Sequence To Sequence (Seq2seq)
 - ❑ Language Translation
 - ❑ Chat Robot

VQA



What's the
mustache made of ?

AI System

Banana

Image credit to <http://www.visualqa.org/challenge.html>

VQA

Who is wearing glasses?

man



woman

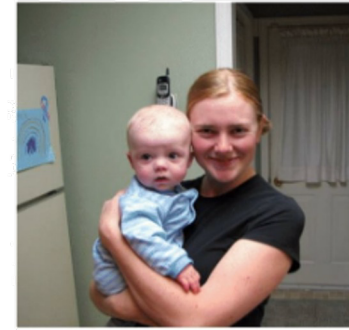


Where is the child sitting?

fridge



arms

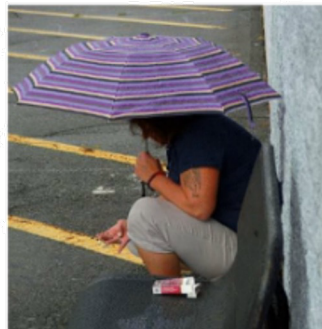


Is the umbrella upside down?

yes



no

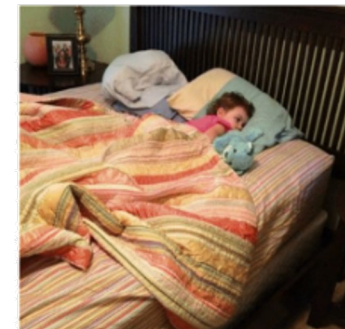


How many children are in the bed?

2

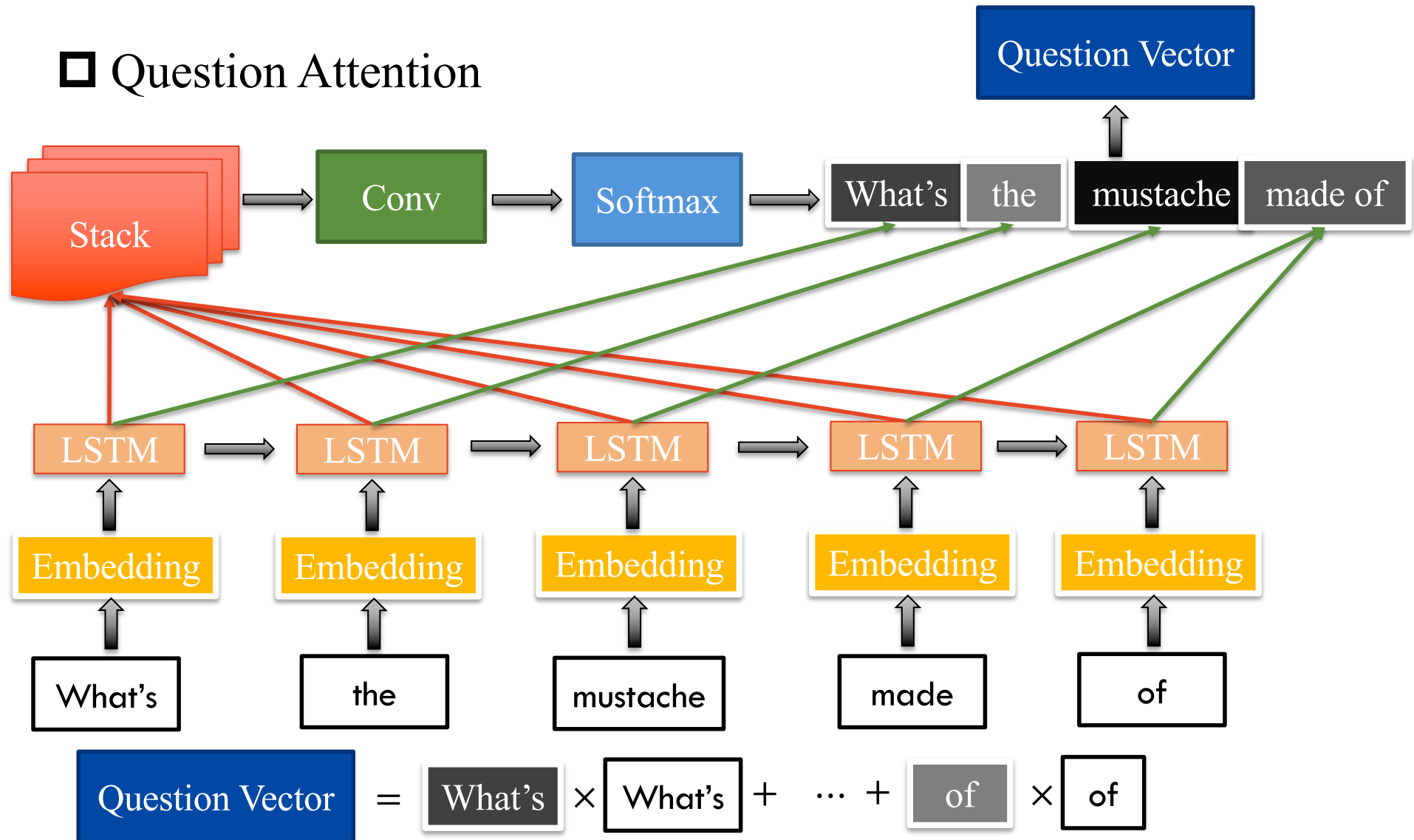


1



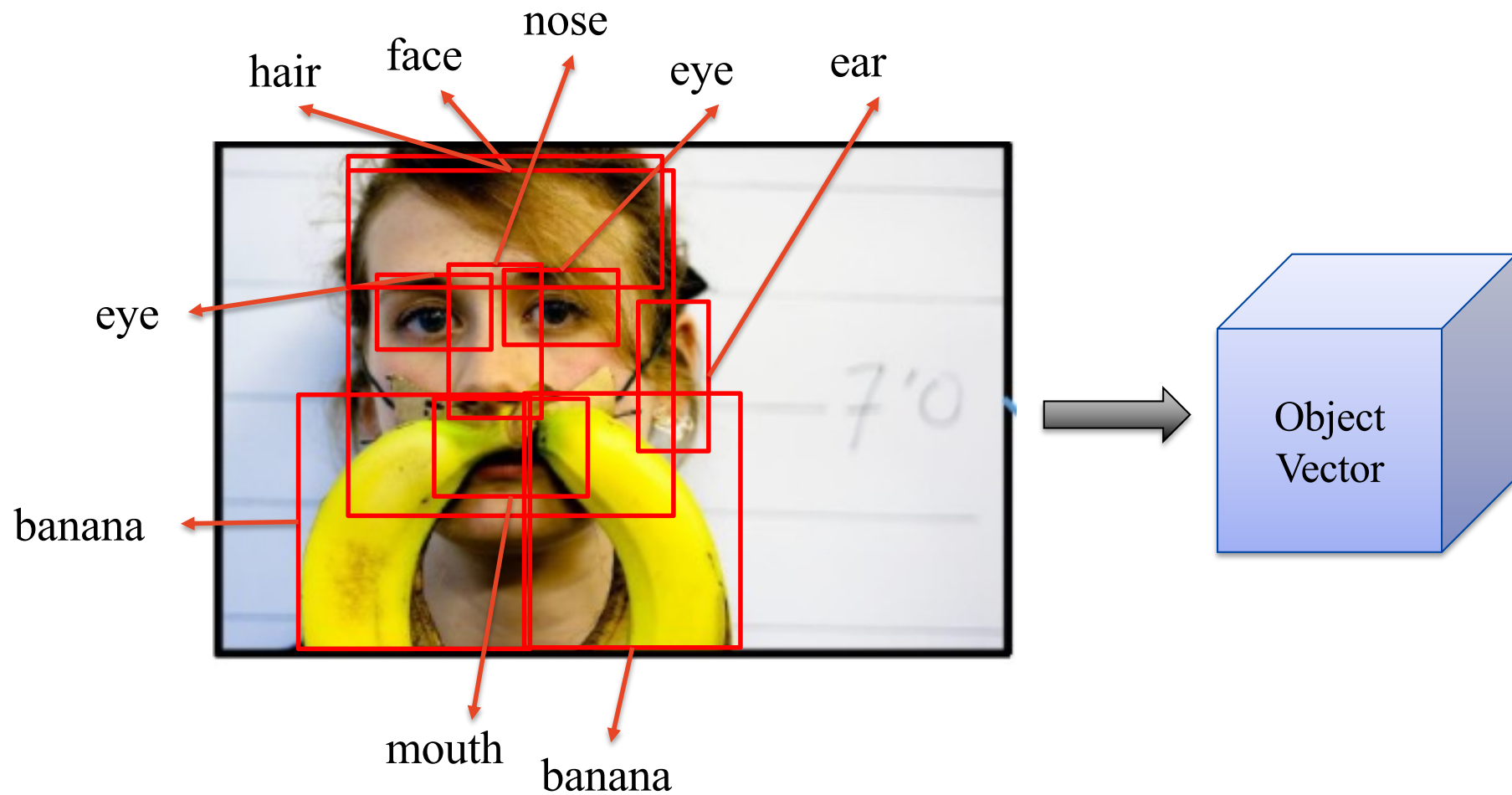
VQA

□ Question Attention



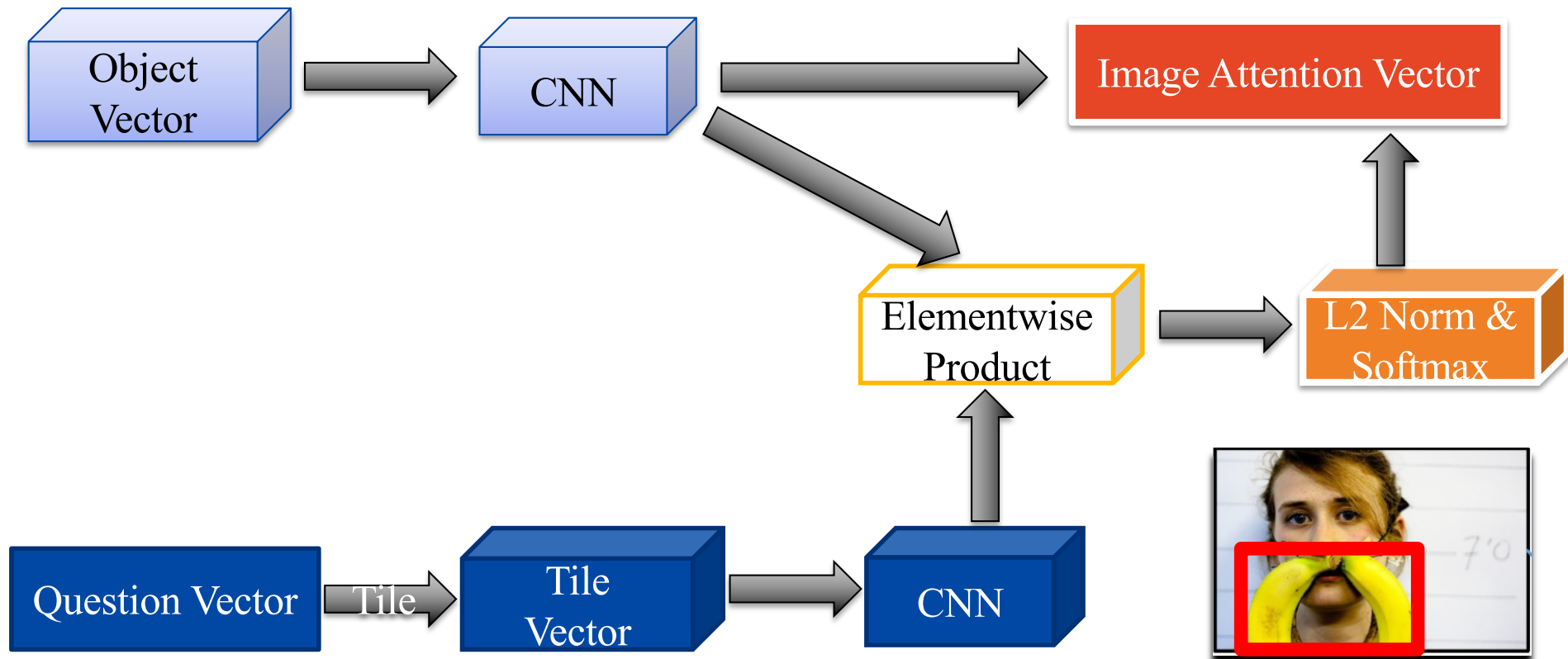
VQA

□ Object Detection



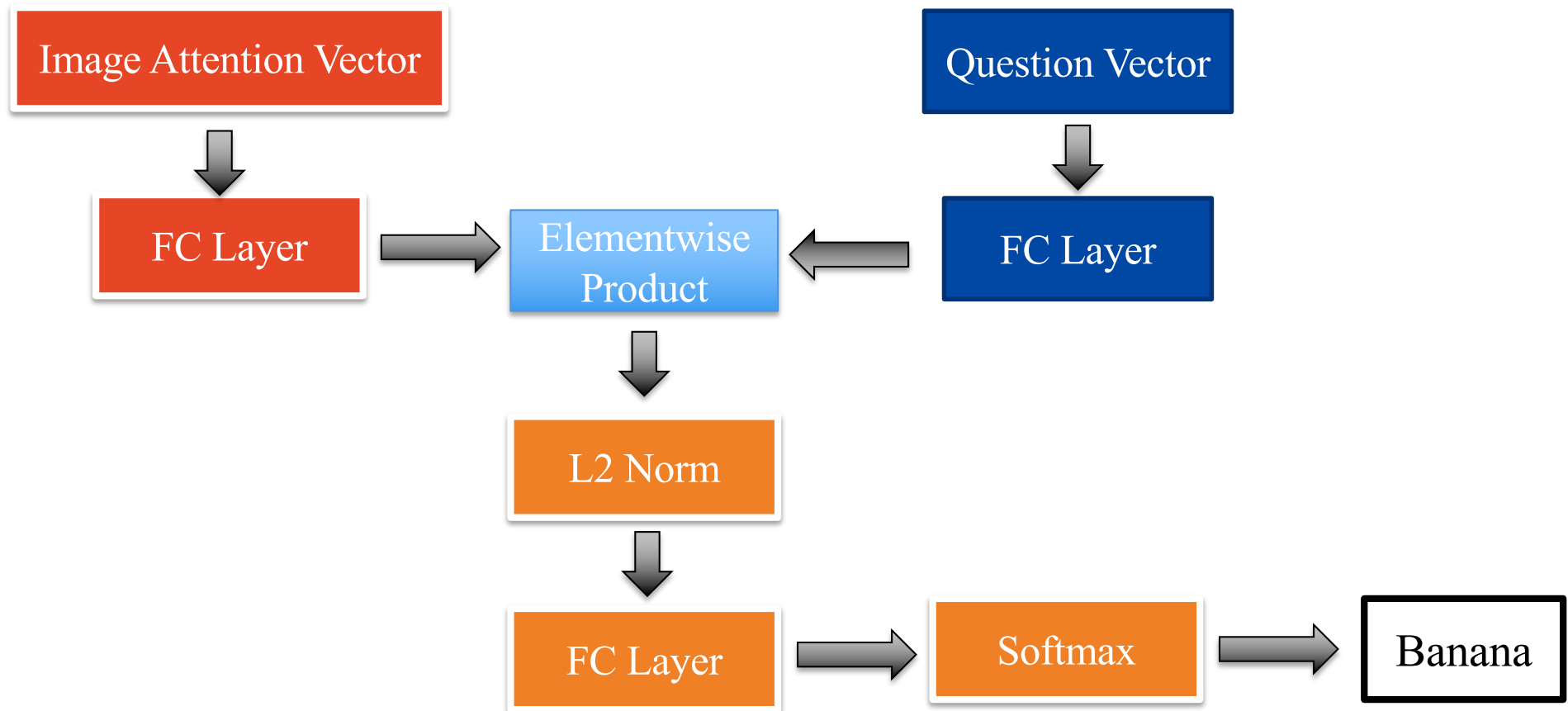
VQA

□ Image Attention



VQA

– Question & Image Fusion



Reading Comprehension

❑ Answering question based on given facts

❑ Example

A. Brian is a frog.

B. Lily is gray.

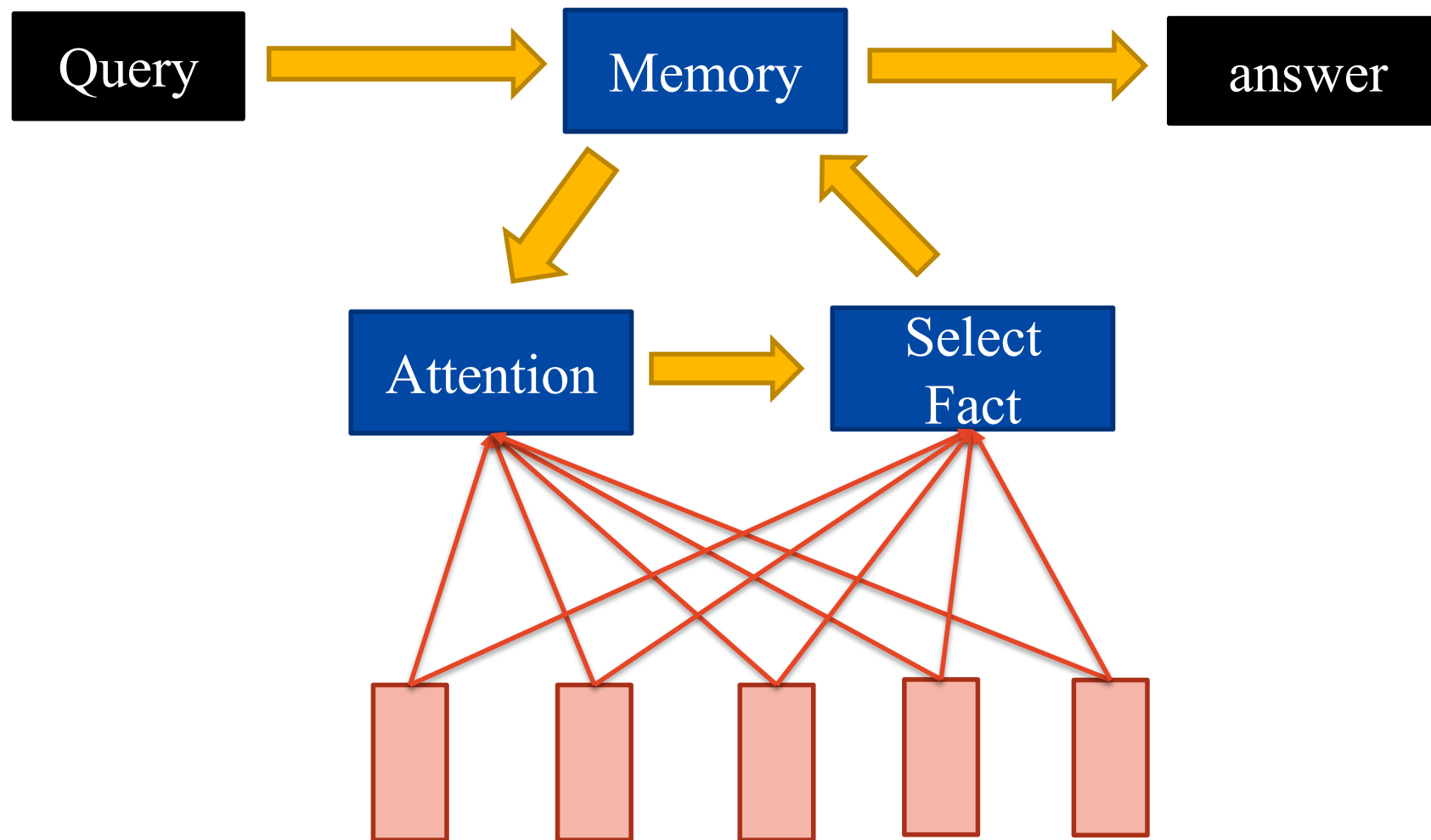
C. Brian is yellow

D. Julius is green.

E. Greg is a frog

✓ What color is Greg?

Reading Comprehension



Each Fact becomes a vector by LSTM

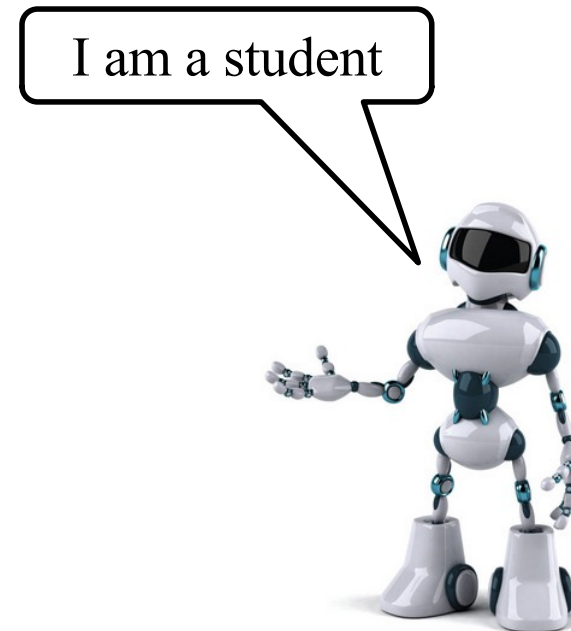
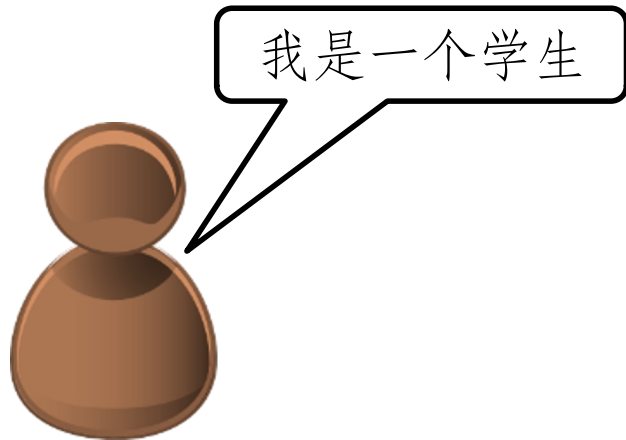
Reading Comprehension

Story (16: basic induction)	Support	Hop 1	Hop 2	Hop 3
Brian is a frog.	yes	0.00	0.98	0.00
Lily is gray.		0.07	0.00	0.00
Brian is yellow.	yes	0.07	0.00	1.00
Julius is green.		0.06	0.00	0.00
Greg is a frog.	yes	0.76	0.02	0.00
What color is Greg? Answer: yellow Prediction: yellow				

End-To-End Memory Networks. S. Sukhbaatar, A. Szlam, J. Weston, R. Fergus. NIPS, 2015.

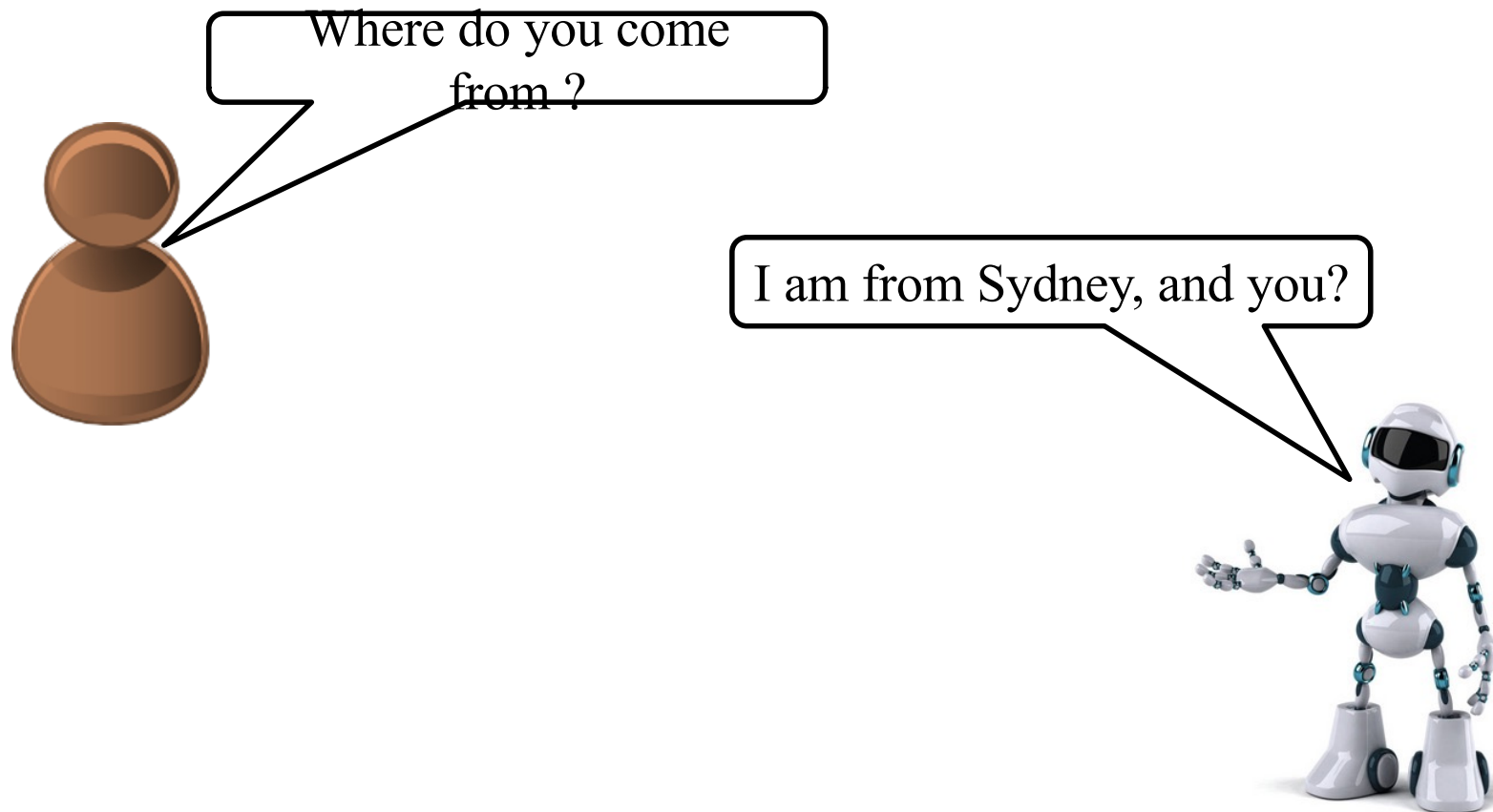
Seq2seq

□ Language Translation



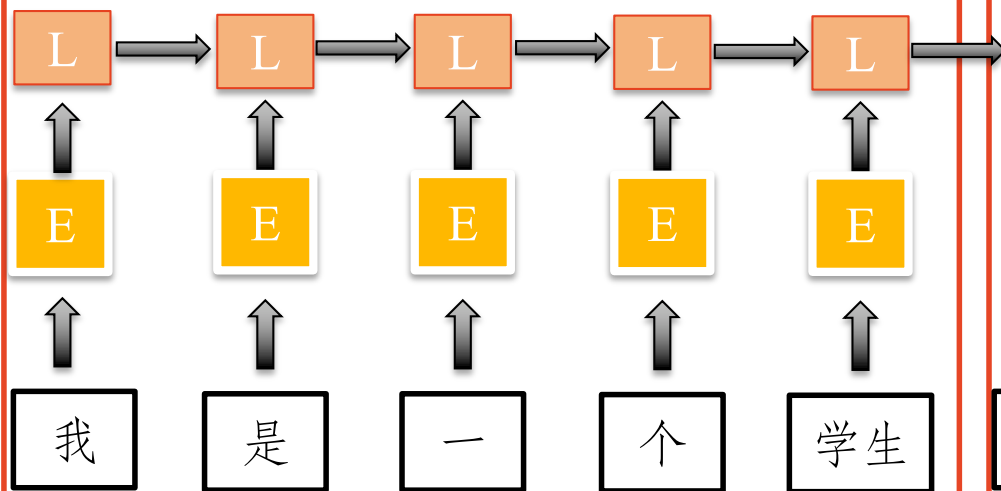
Seq2seq

□ Chat Robot

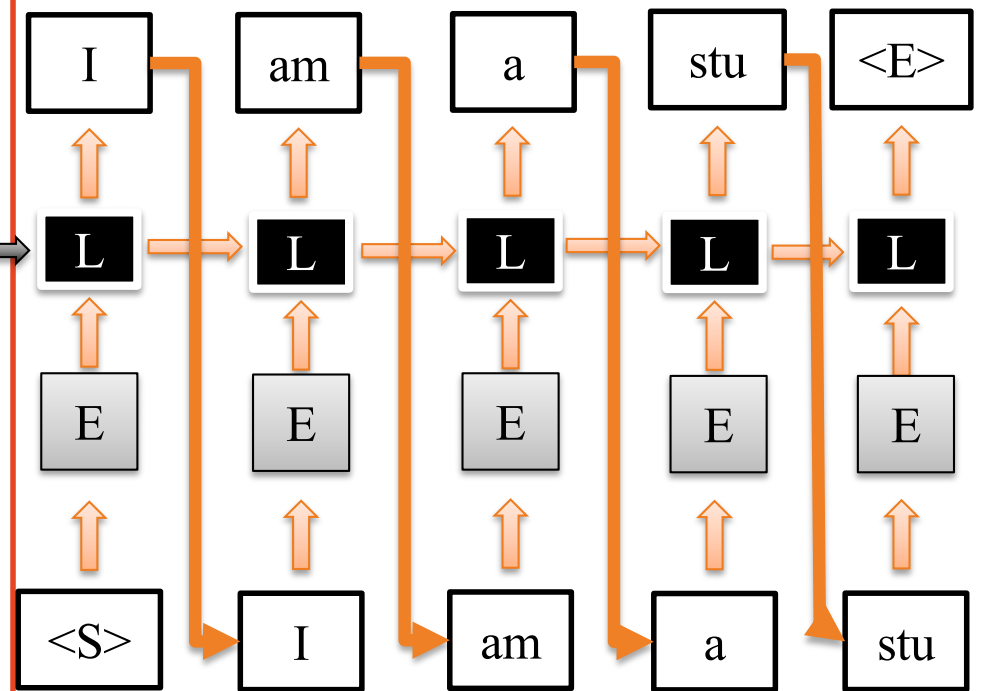


Seq2seq

Encoder

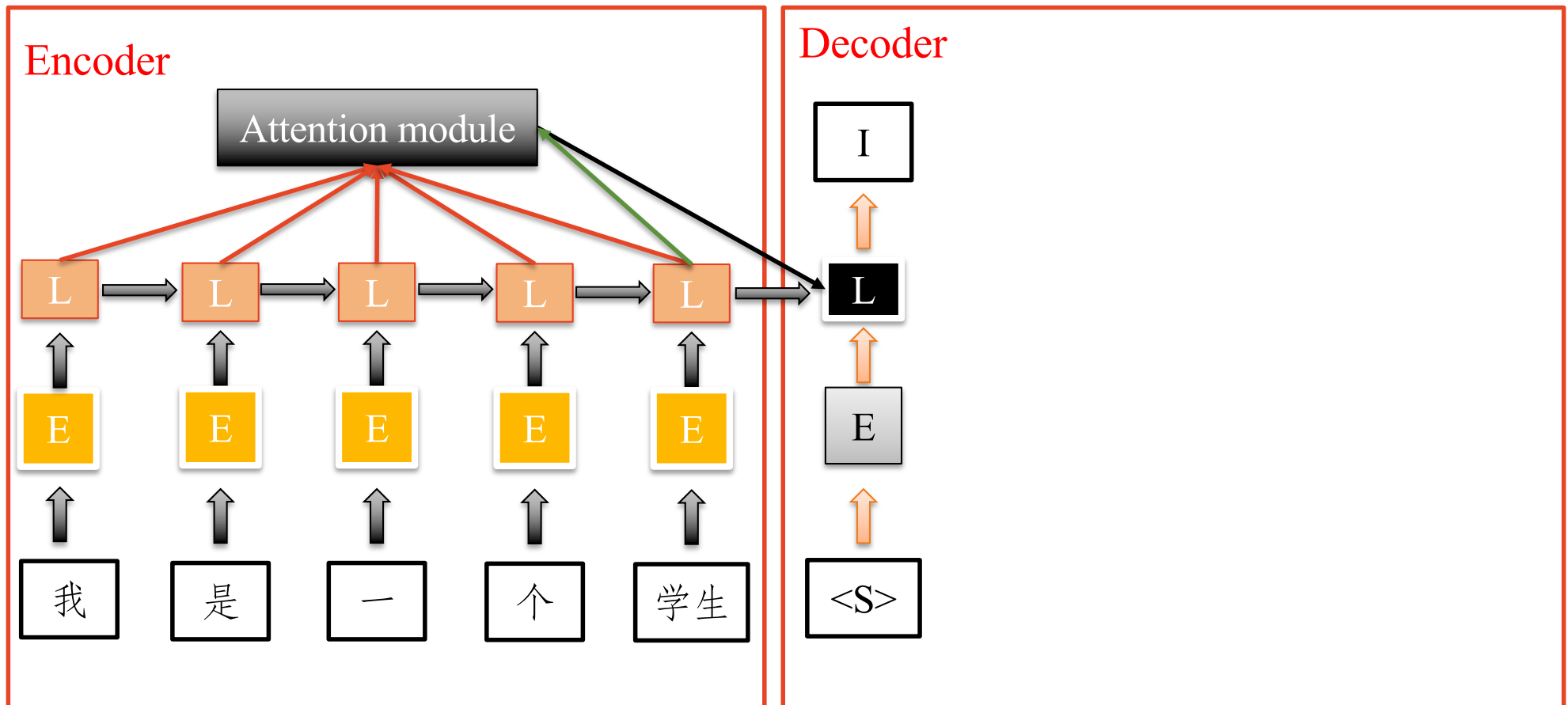


Decoder



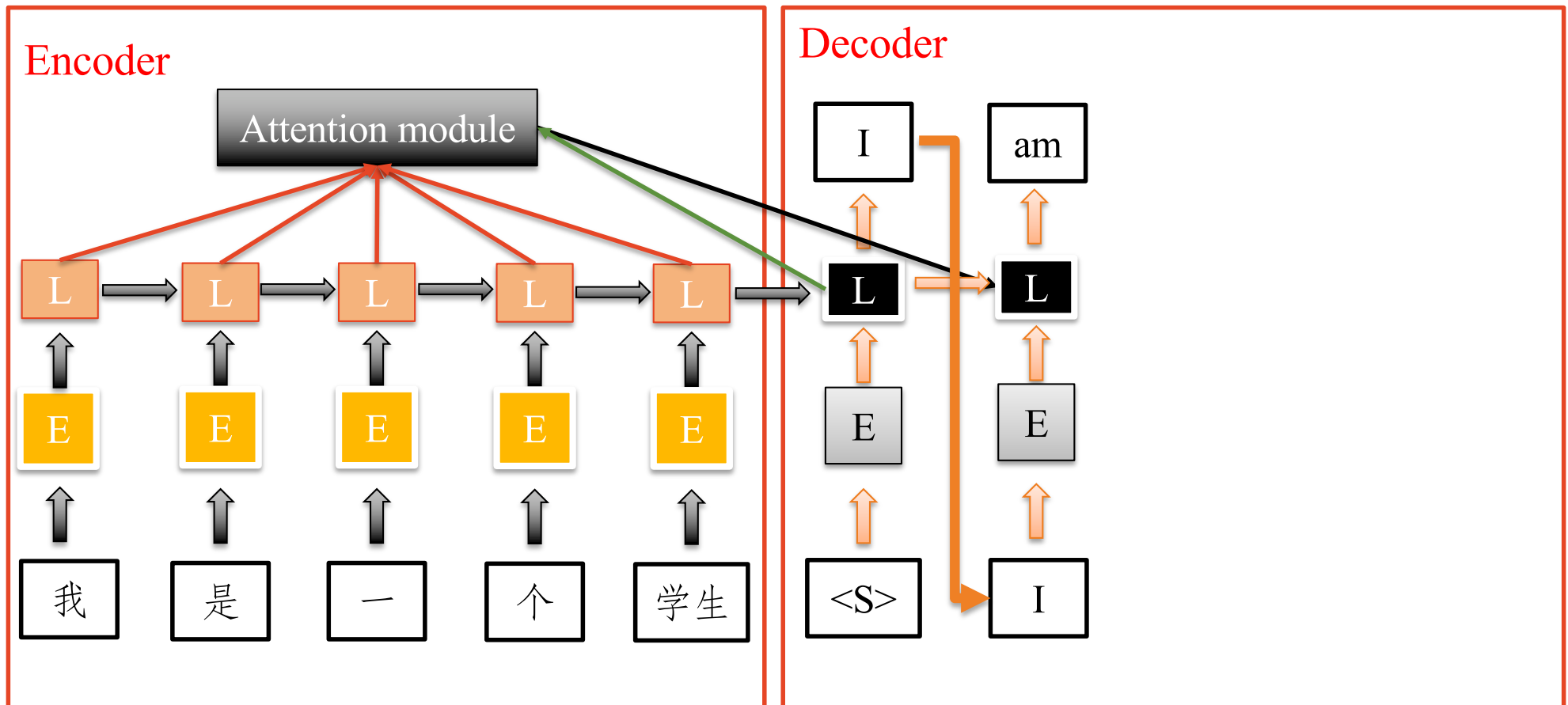
Seq2seq

□ Attention



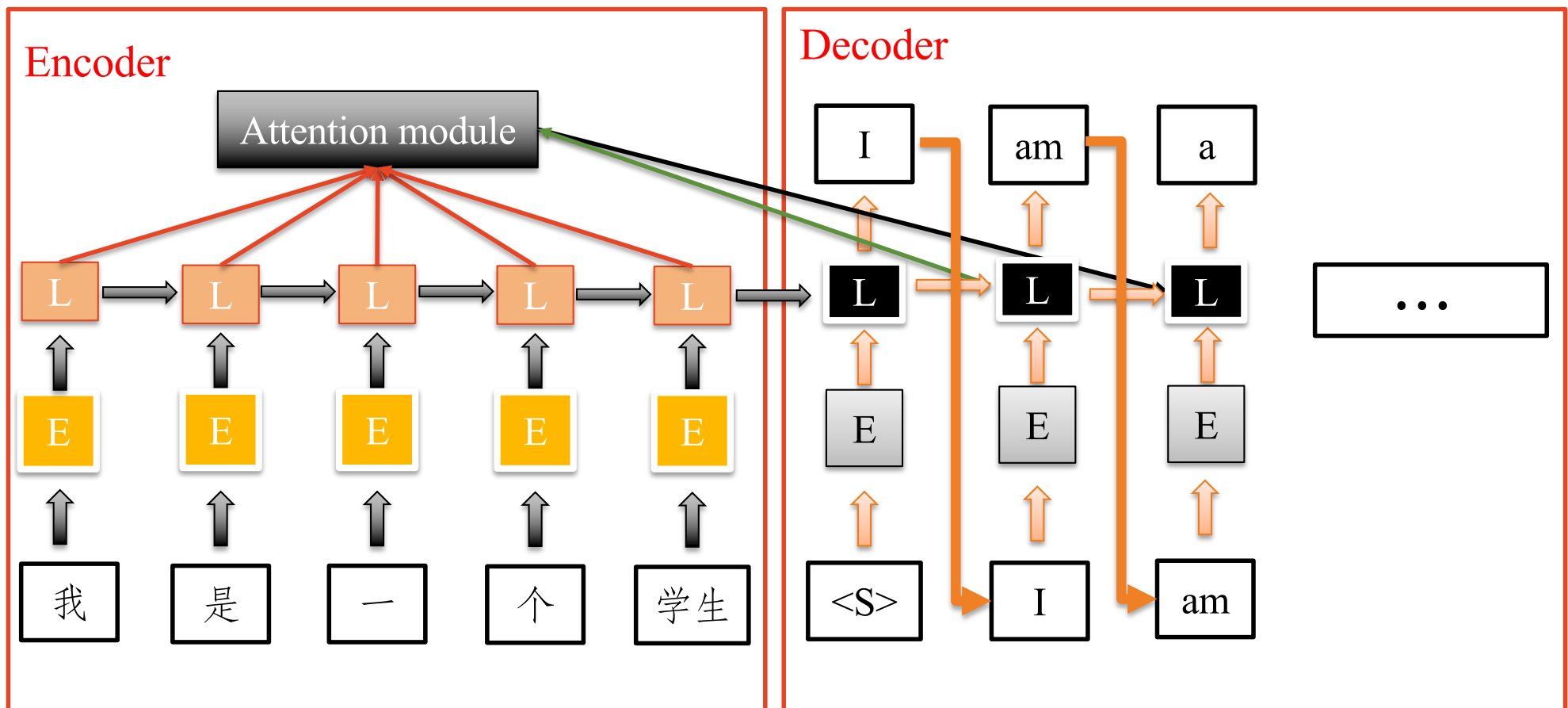
Seq2seq

□ Attention



Seq2seq

□ Attention



Seq2seq

- ❑ Two stage
 - ❑ Encode stage
 - ❑ Decode stage
- ❑ Attention
 - ❑ Weight for each encode symbol in decode step
 - ❑ Another input of LSTM represented as a_t
 - ❑ $f_t = \sigma(w_{fh}h_{t-1} + w_{fx}x_t + w_{fa}a_t + b_f)$
 - ❑ $i_t = \sigma(w_{ih}h_{t-1} + w_{ix}x_t + w_{ia}a_t + b_i)$
 - ❑ $o_t = \sigma(w_{oh}h_{t-1} + w_{ox}x_t + w_{oa}a_t + b_o)$
 - ❑ $\tilde{c}_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + w_{ha}a_t + b_h)$

Seq2seq

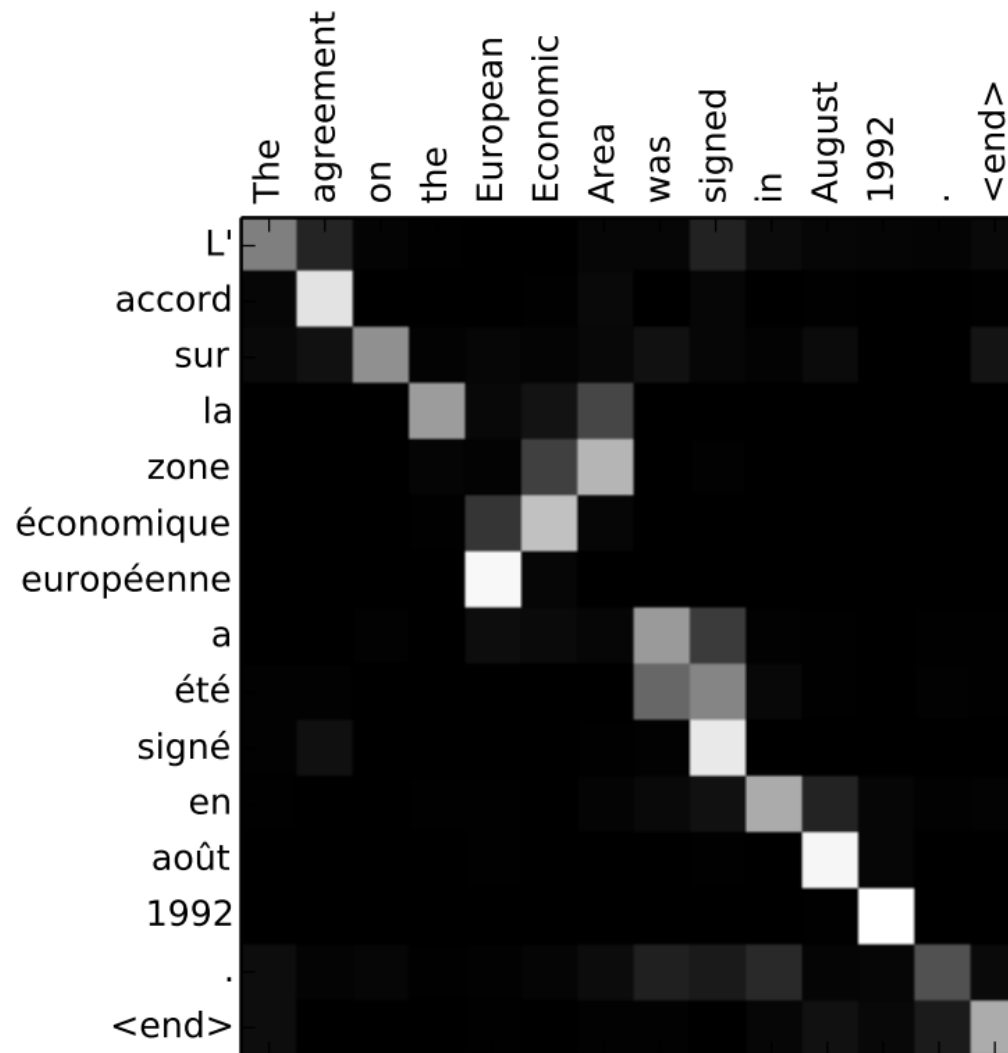
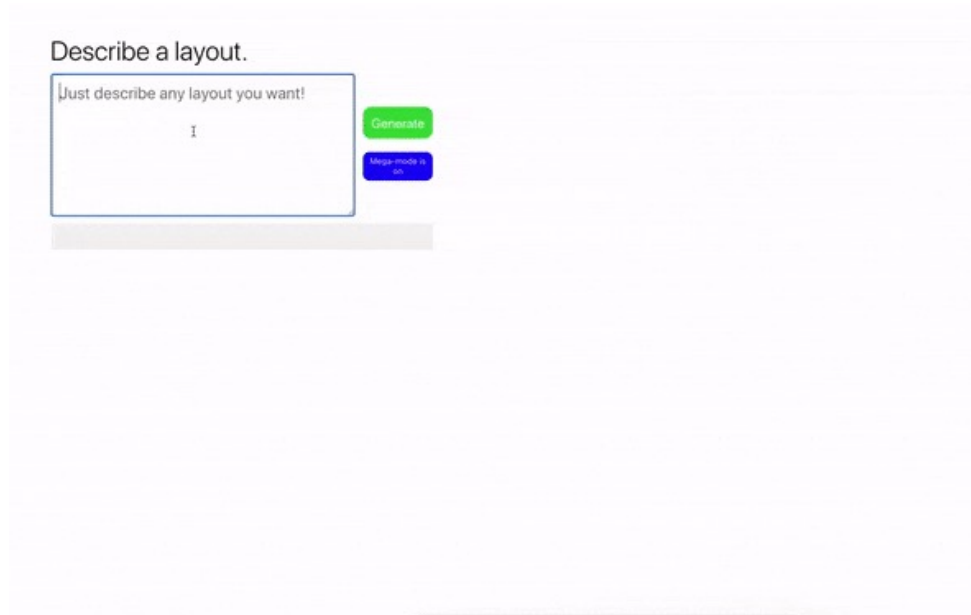


Image credit to Neural Machine Translation by
Jointly Learning to Align and Translate

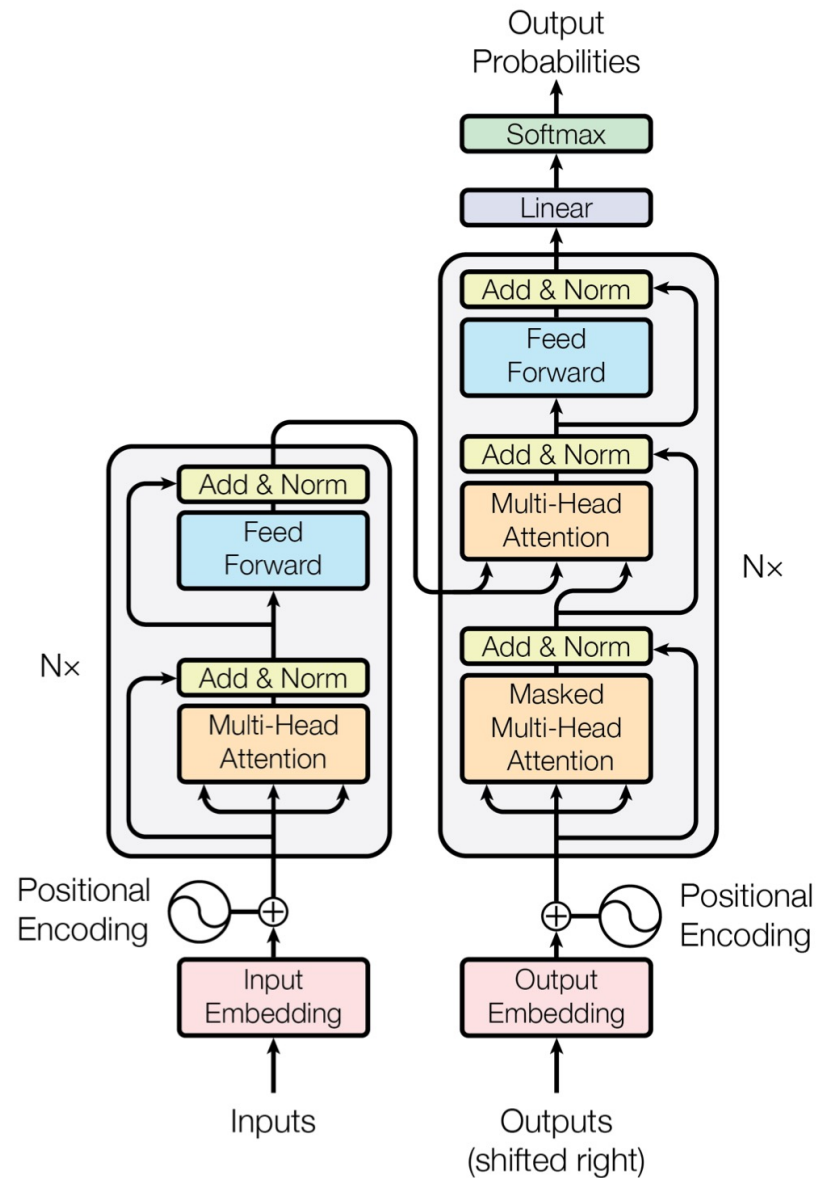
Transformer

AI's Big Breakthroughs

GPT-3, the 3rd version of *Generative Pre-Trained Transformer* model released by OpenAI.

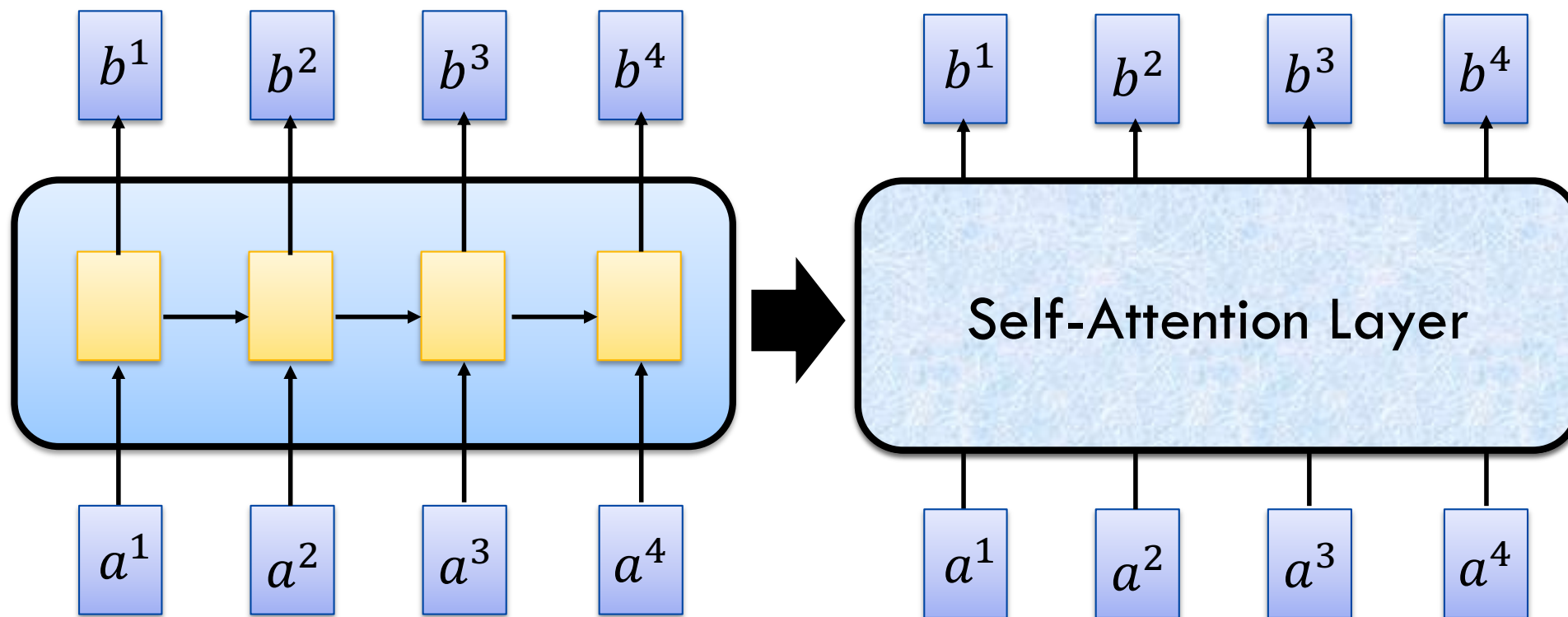


Develop Web apps. Enter a sentence describing Google home page layout, and here you see GPT-3 generating the code for



Self-Attention

Self-Attention



You can try to replace any thing that has been done by RNN with self-attention.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

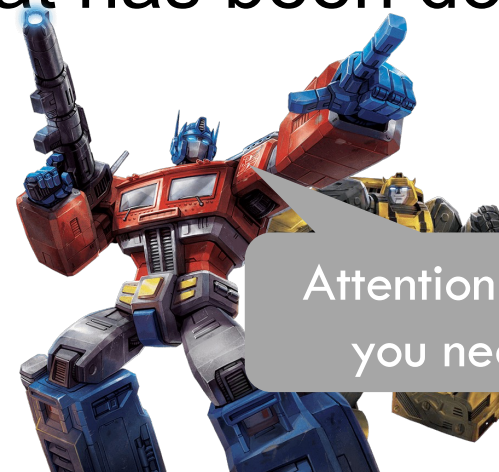
Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

The University of Sydney

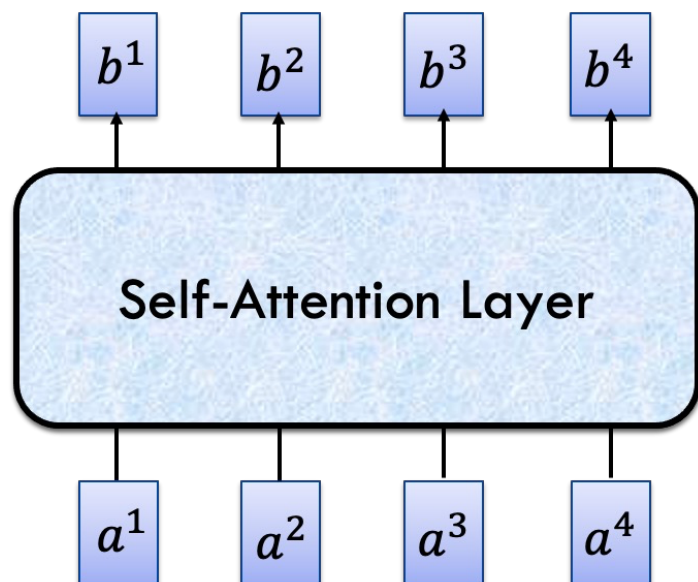
6 Dec 2017



Attention is all
you need.

Page 56

Self-Attention



q : **query** (to match others)

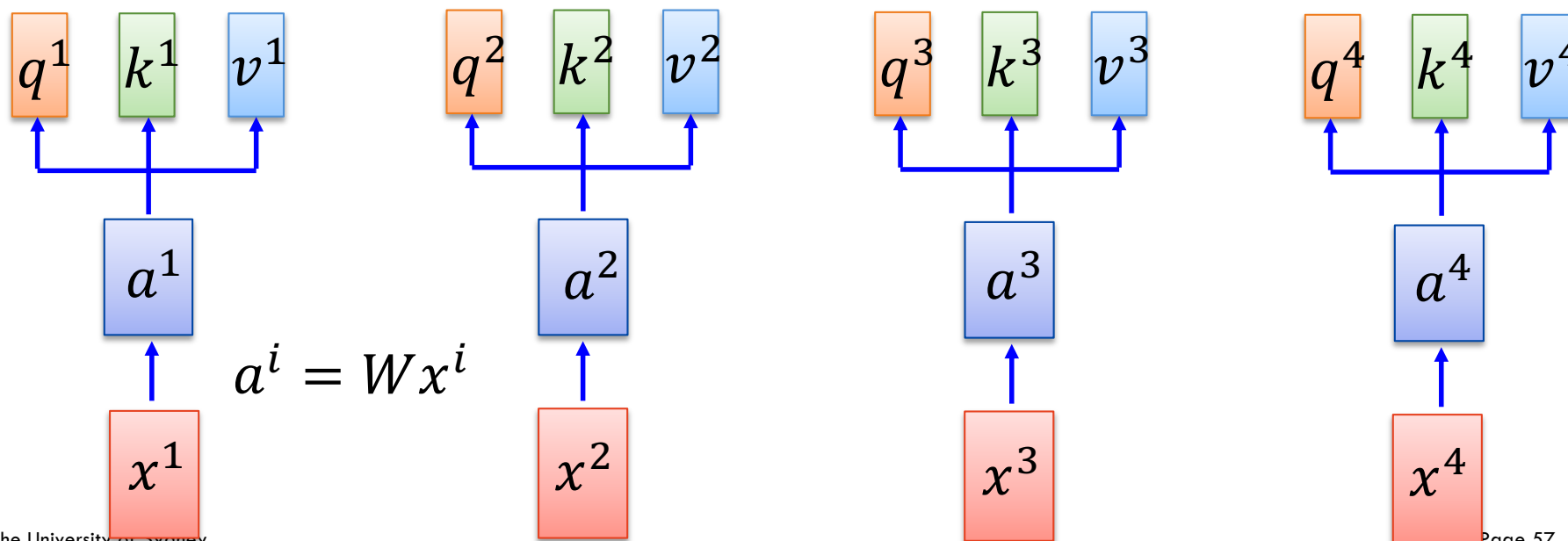
$$q^i = W^q a^i$$

k : **key** (to be matched)

$$k^i = W^k a^i$$

v : **value** (information to be extracted)

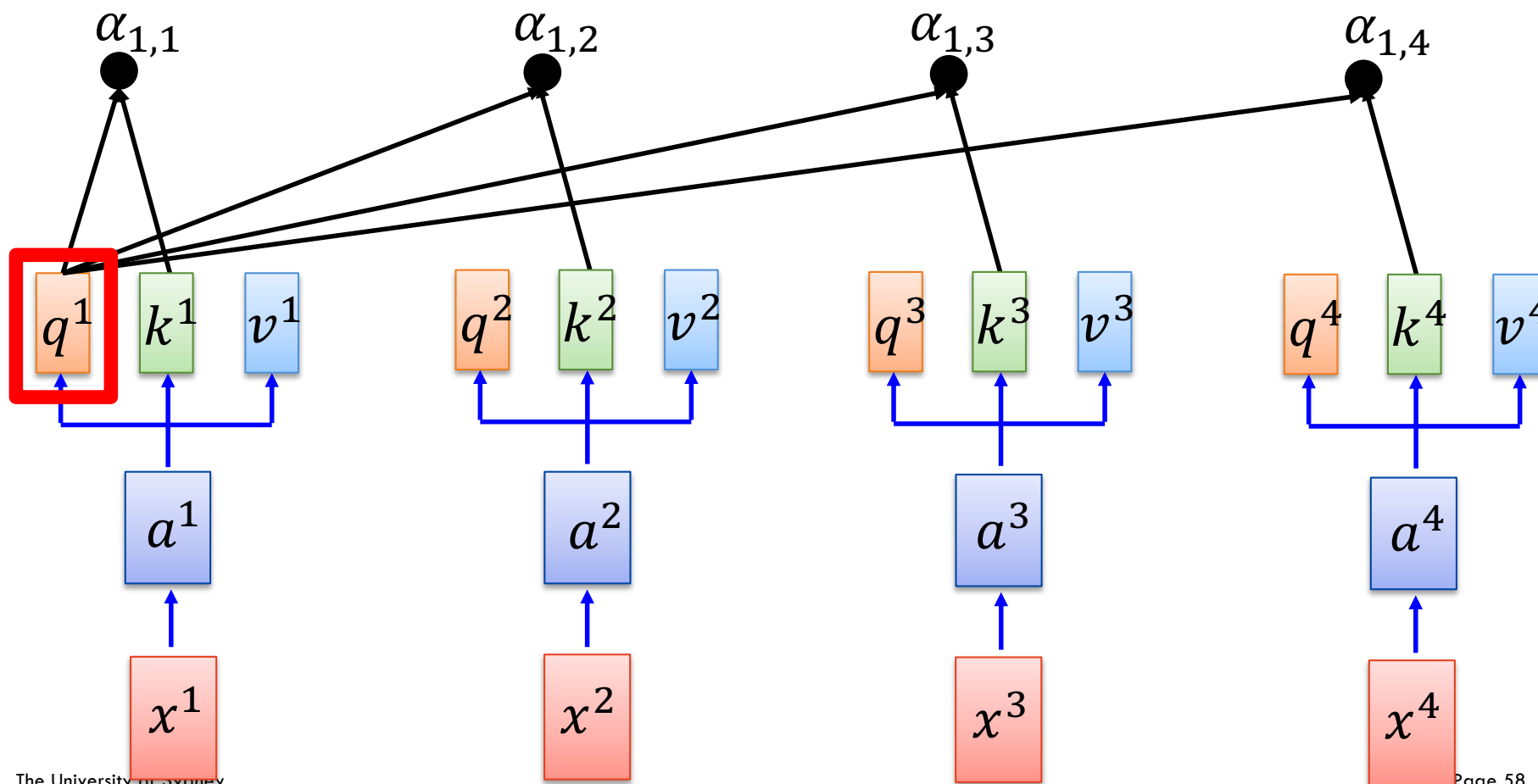
$$v^i = W^v a^i$$



Self-Attention

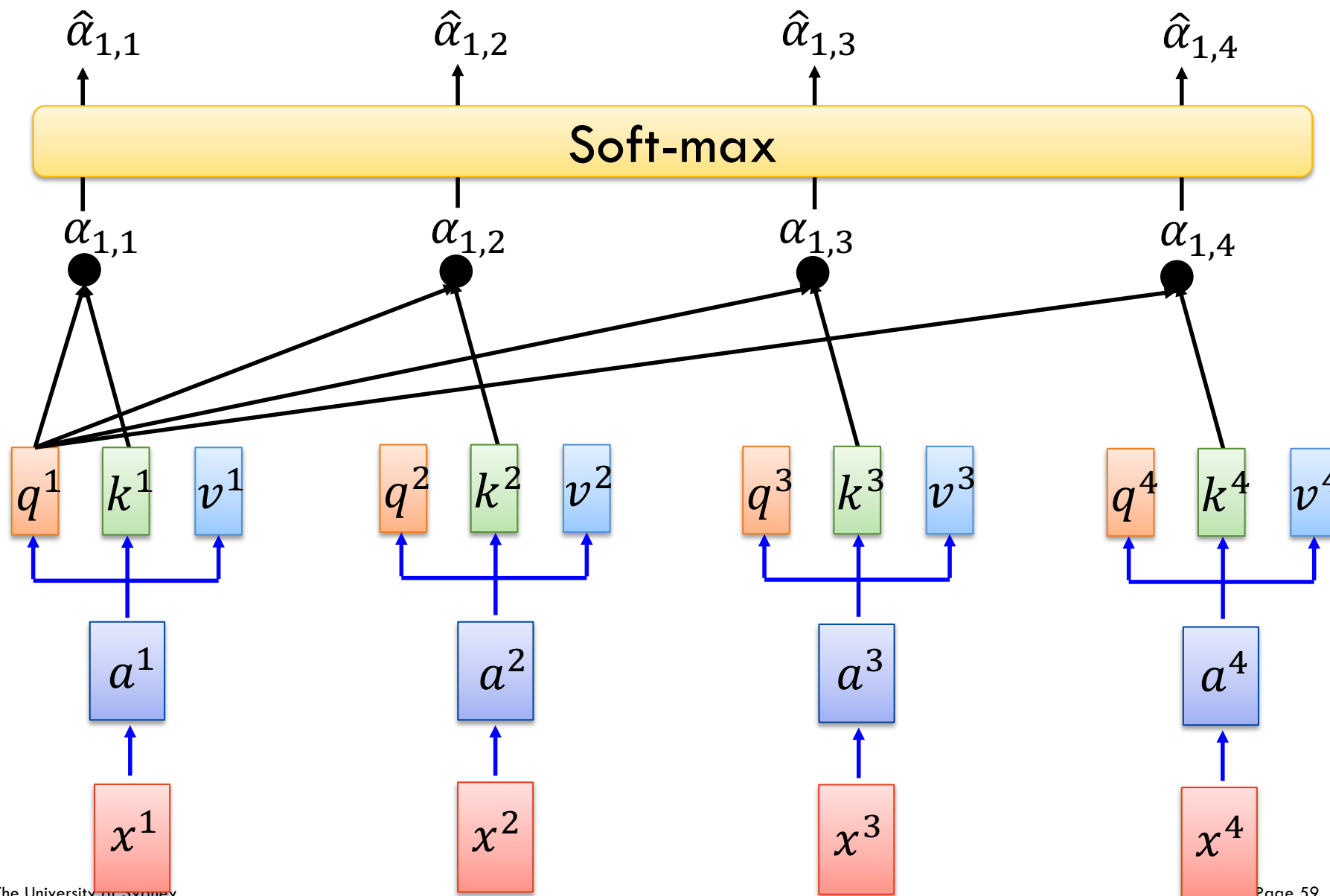
Scaled Dot-Product Attention: $\alpha_{1,i} = \underbrace{q^1 \cdot k^i}_{\text{dot product}} / \sqrt{d}$

d is the dim of q and k



Self-Attention

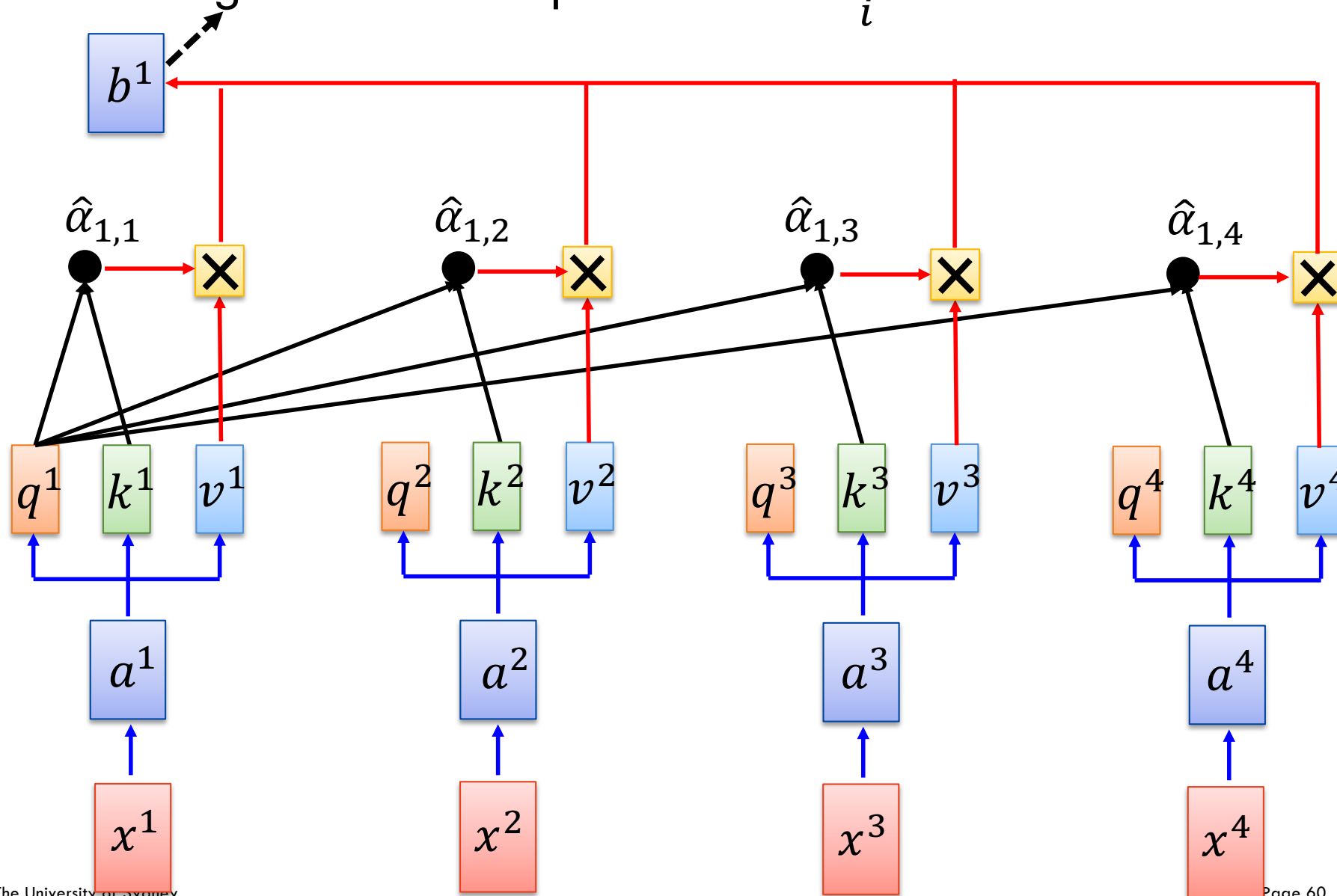
$$\hat{\alpha}_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$



Self-Attention

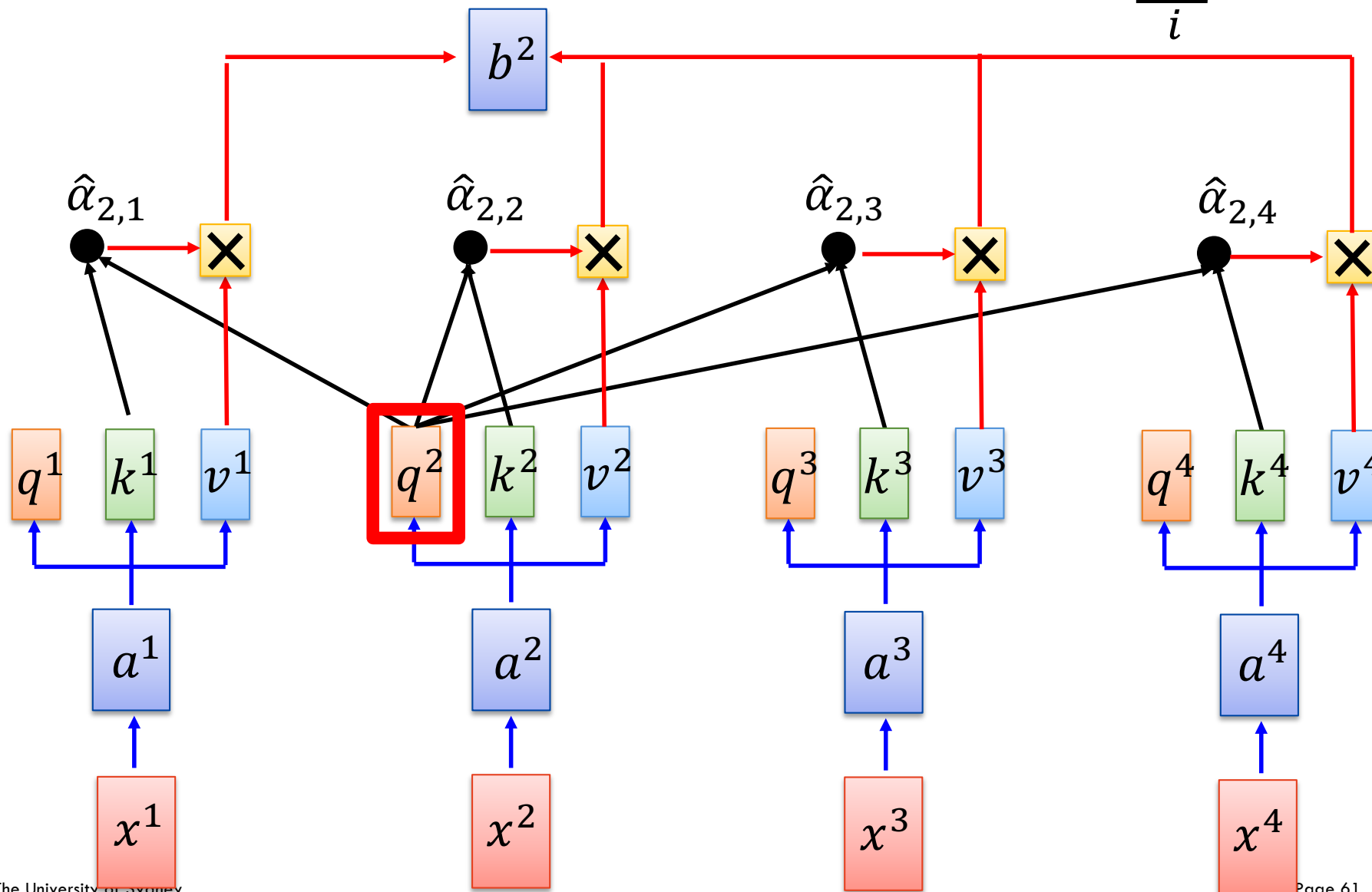
Considering the whole sequence

$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$



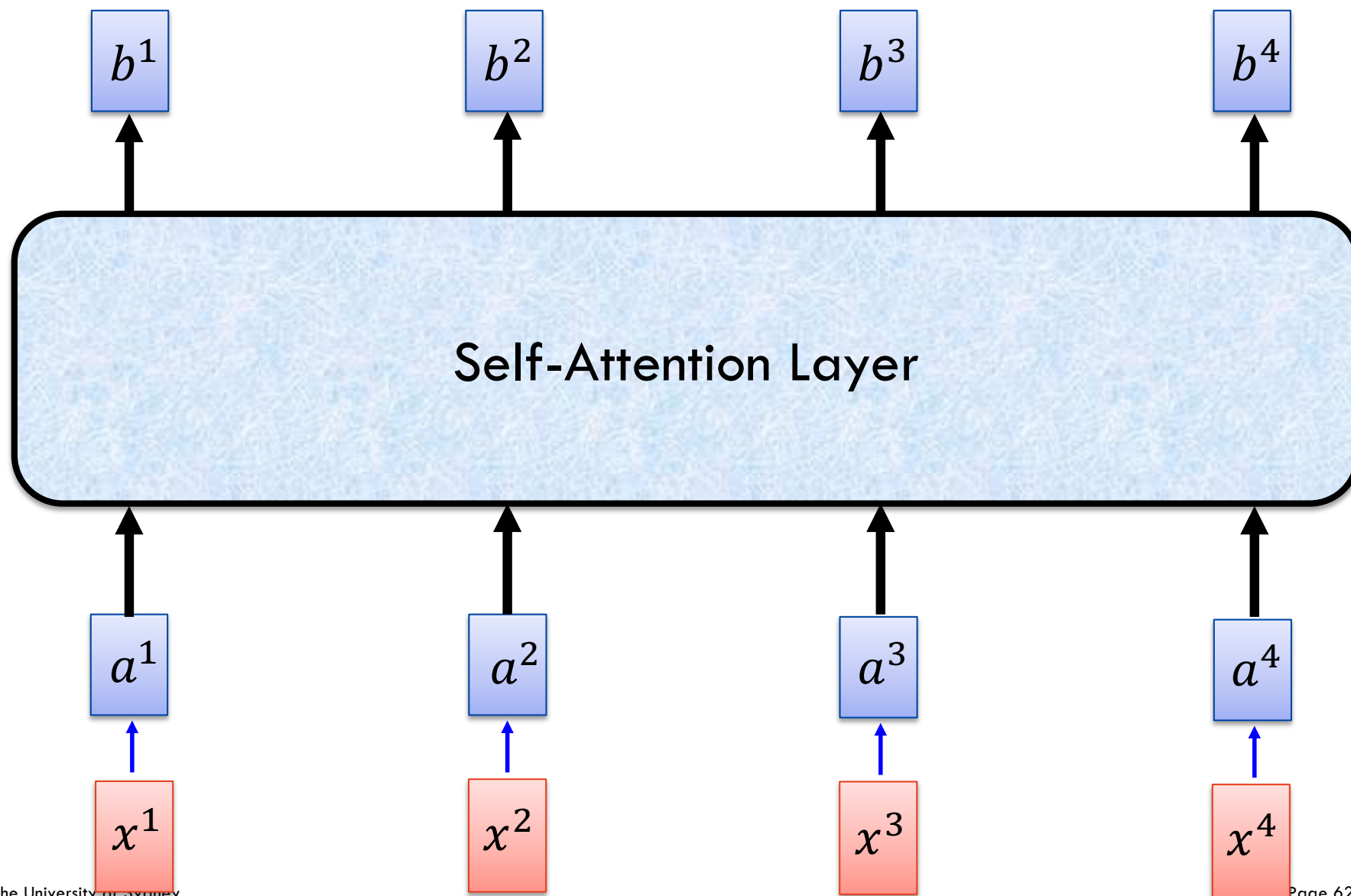
Self-Attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



Self-Attention

b^1, b^2, b^3, b^4 can be parallelly computed.



Self-Attention

$$q^i = W^q a^i$$

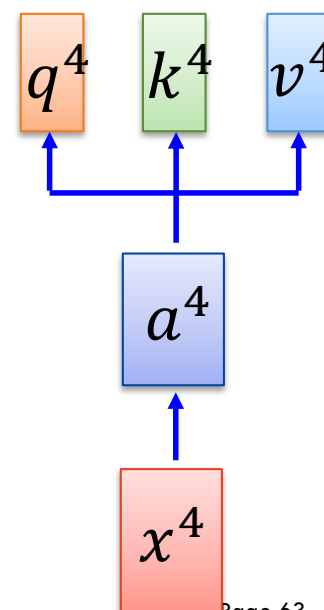
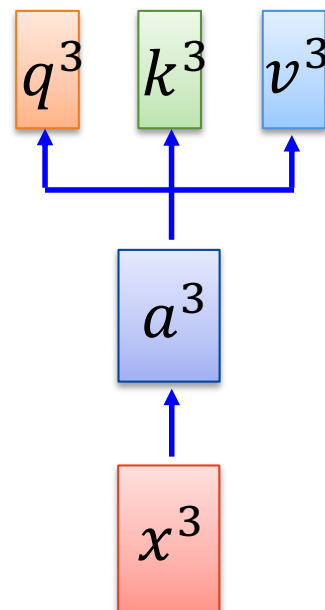
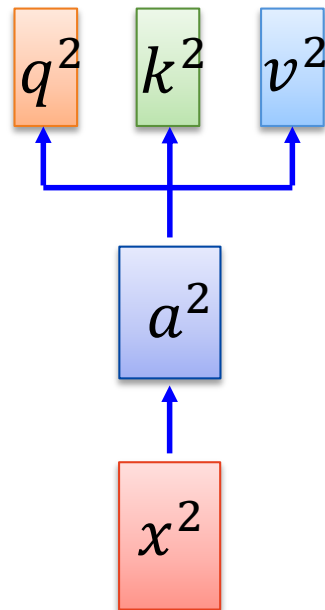
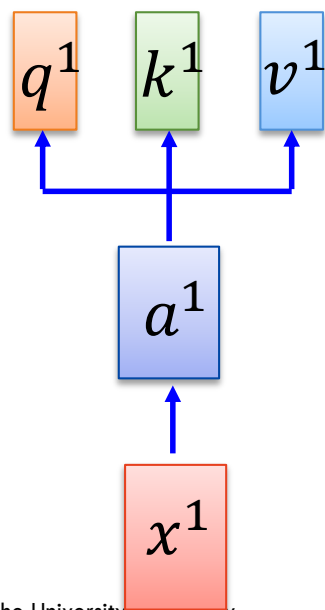
$$k^i = W^k a^i$$

$$v^i = W^v a^i$$

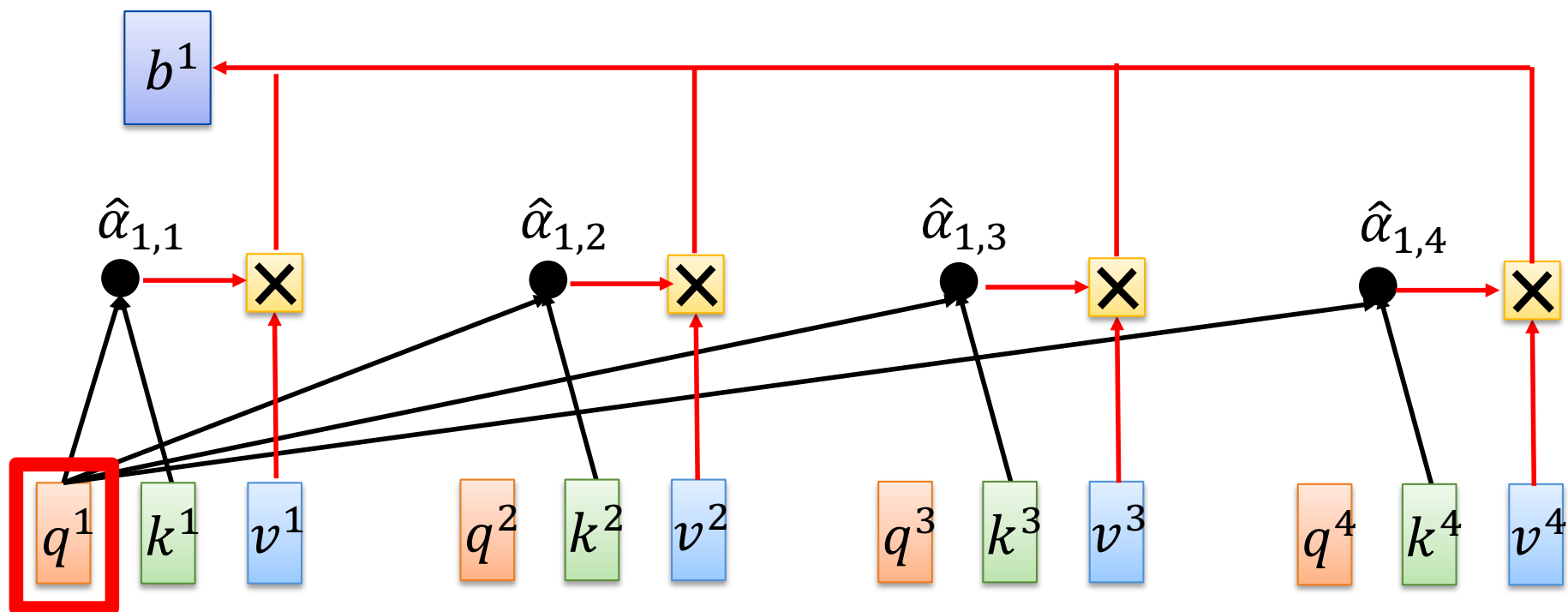
$$\begin{matrix} q^1 & q^2 & q^3 & q^4 \\ \hline Q \end{matrix} = \begin{matrix} W^q & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix} \end{matrix}$$

$$\begin{matrix} k^1 & k^2 & k^3 & k^4 \\ \hline K \end{matrix} = \begin{matrix} W^k & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix} \end{matrix}$$

$$\begin{matrix} v^1 & v^2 & v^3 & v^4 \\ \hline V \end{matrix} = \begin{matrix} W^v & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix} \end{matrix}$$



Self-Attention



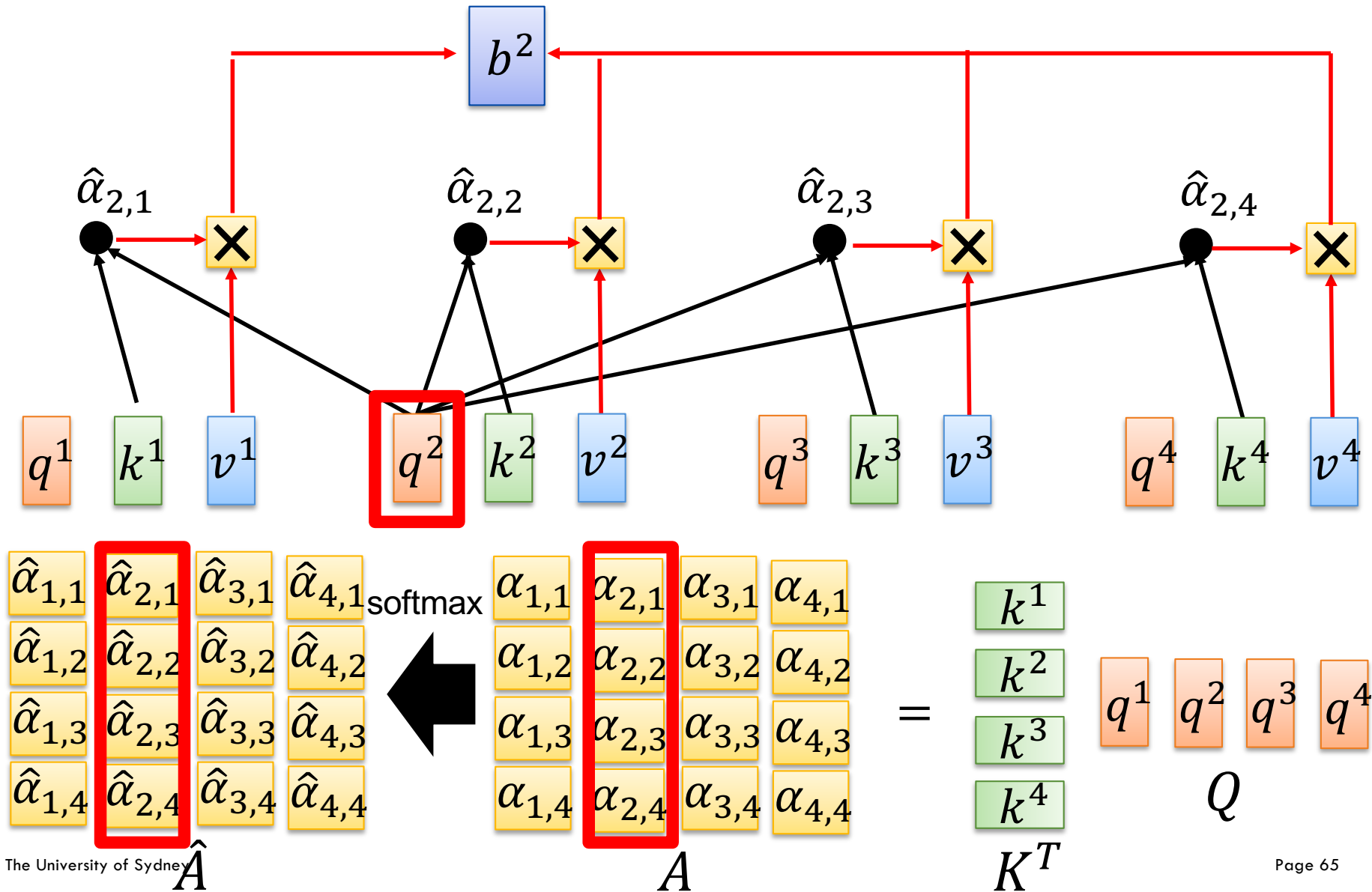
$$\begin{aligned} \alpha_{1,1} &= k^1 q^1 & \alpha_{1,2} &= k^2 q^1 \\ \alpha_{1,3} &= k^3 q^1 & \alpha_{1,4} &= k^4 q^1 \end{aligned}$$

$$\begin{bmatrix} \alpha_{1,1} \\ \alpha_{1,2} \\ \alpha_{1,3} \\ \alpha_{1,4} \end{bmatrix} = \begin{bmatrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{bmatrix} q^1$$

(ignore $\sqrt{d}$ for simplicity)

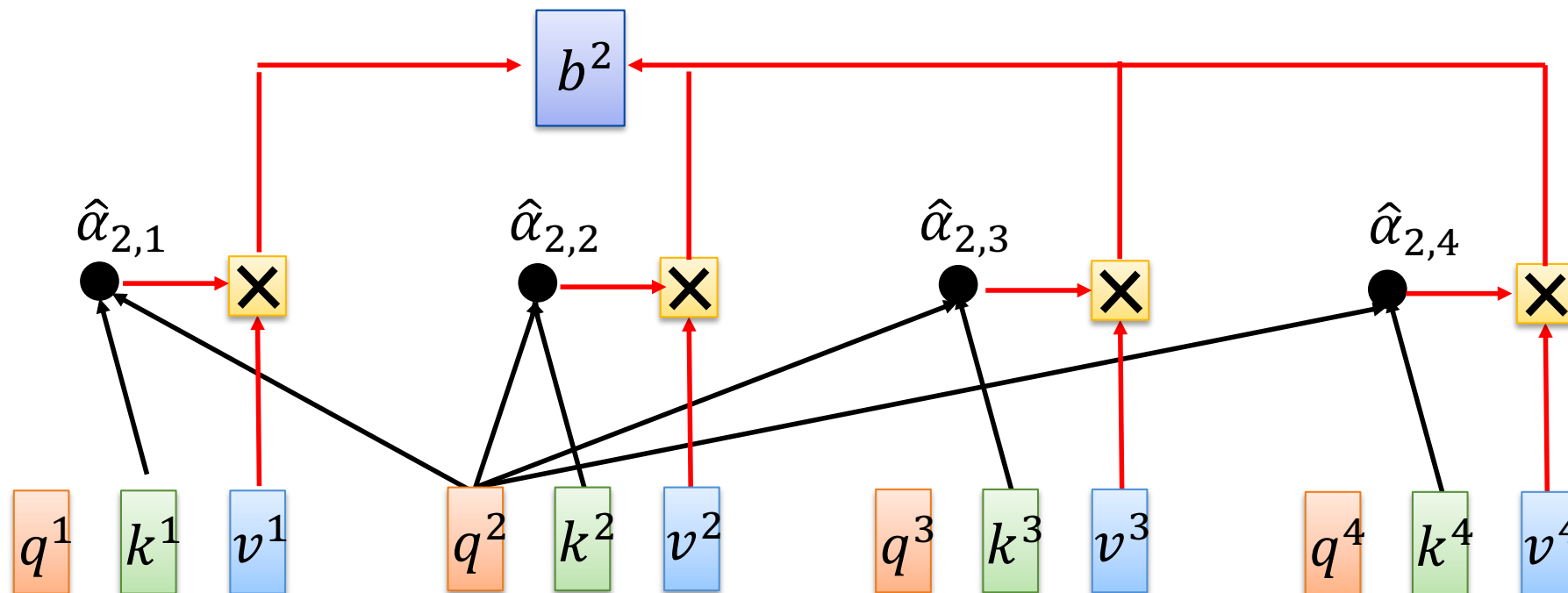
Self-Attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



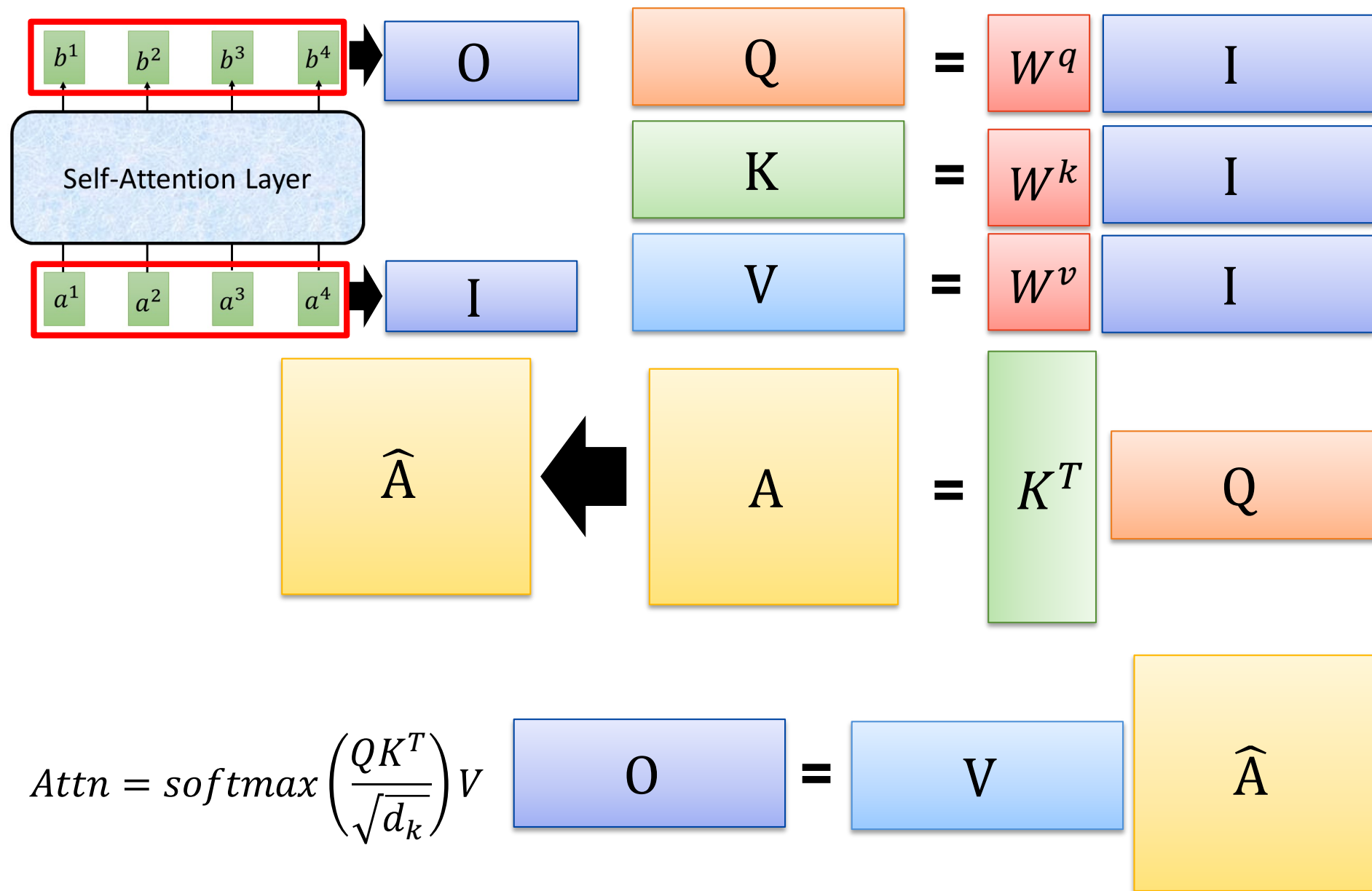
Self-Attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



$$\begin{matrix} b^1 & b^2 & b^3 & b^4 \\ \hline O \end{matrix} = \begin{matrix} v^1 & v^2 & v^3 & v^4 \\ \hline V \end{matrix} \begin{matrix} \hat{\alpha}_{1,1} & \hat{\alpha}_{2,1} & \hat{\alpha}_{3,1} & \hat{\alpha}_{4,1} \\ \hat{\alpha}_{1,2} & \hat{\alpha}_{2,2} & \hat{\alpha}_{3,2} & \hat{\alpha}_{4,2} \\ \hat{\alpha}_{1,3} & \hat{\alpha}_{2,3} & \hat{\alpha}_{3,3} & \hat{\alpha}_{4,3} \\ \hat{\alpha}_{1,4} & \hat{\alpha}_{2,4} & \hat{\alpha}_{3,4} & \hat{\alpha}_{4,4} \\ \hline \hat{A} \end{matrix}$$

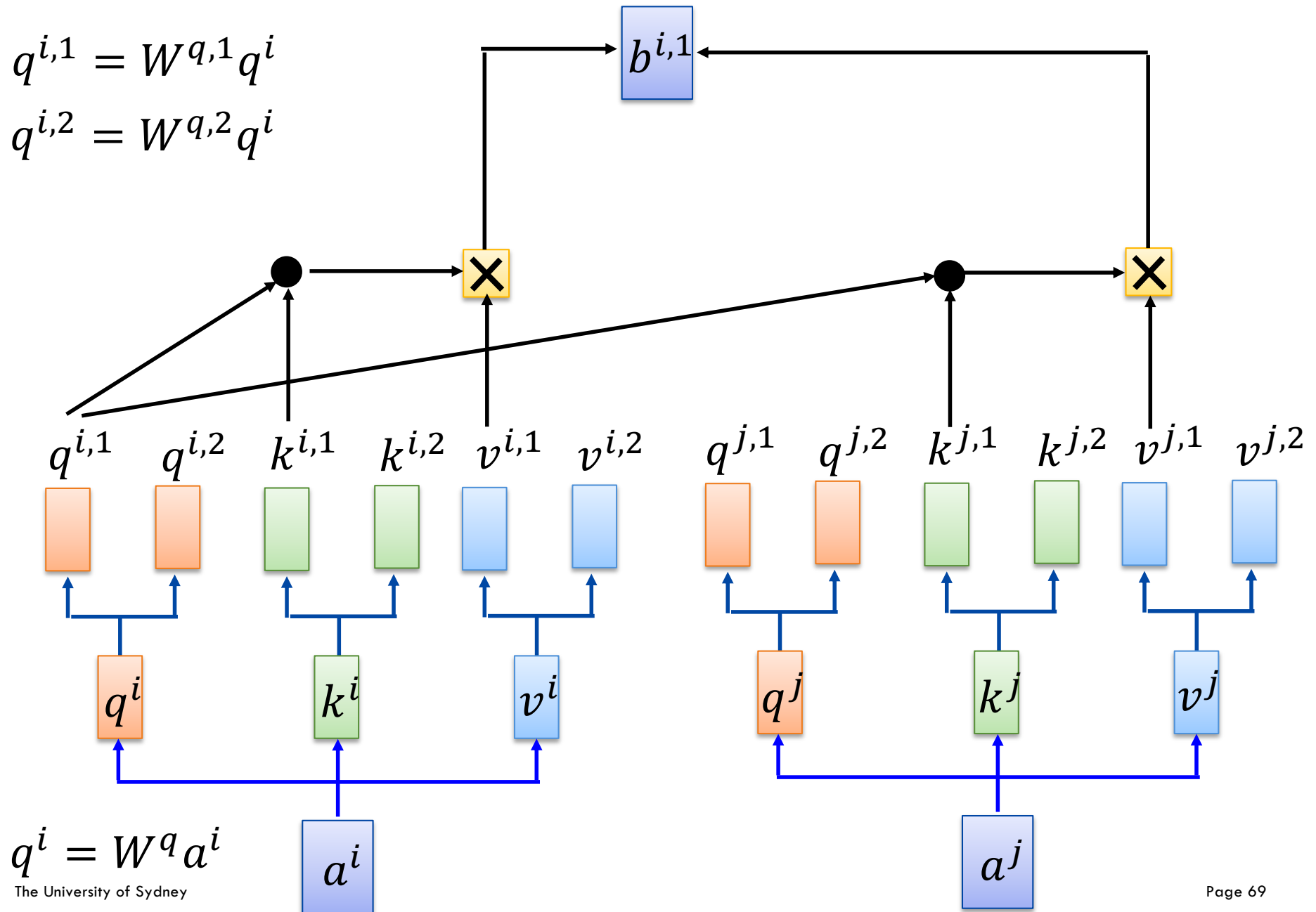
Self-Attention



Multi-head Self-Attention

Multi-head Self-Attention

(2 heads as example)

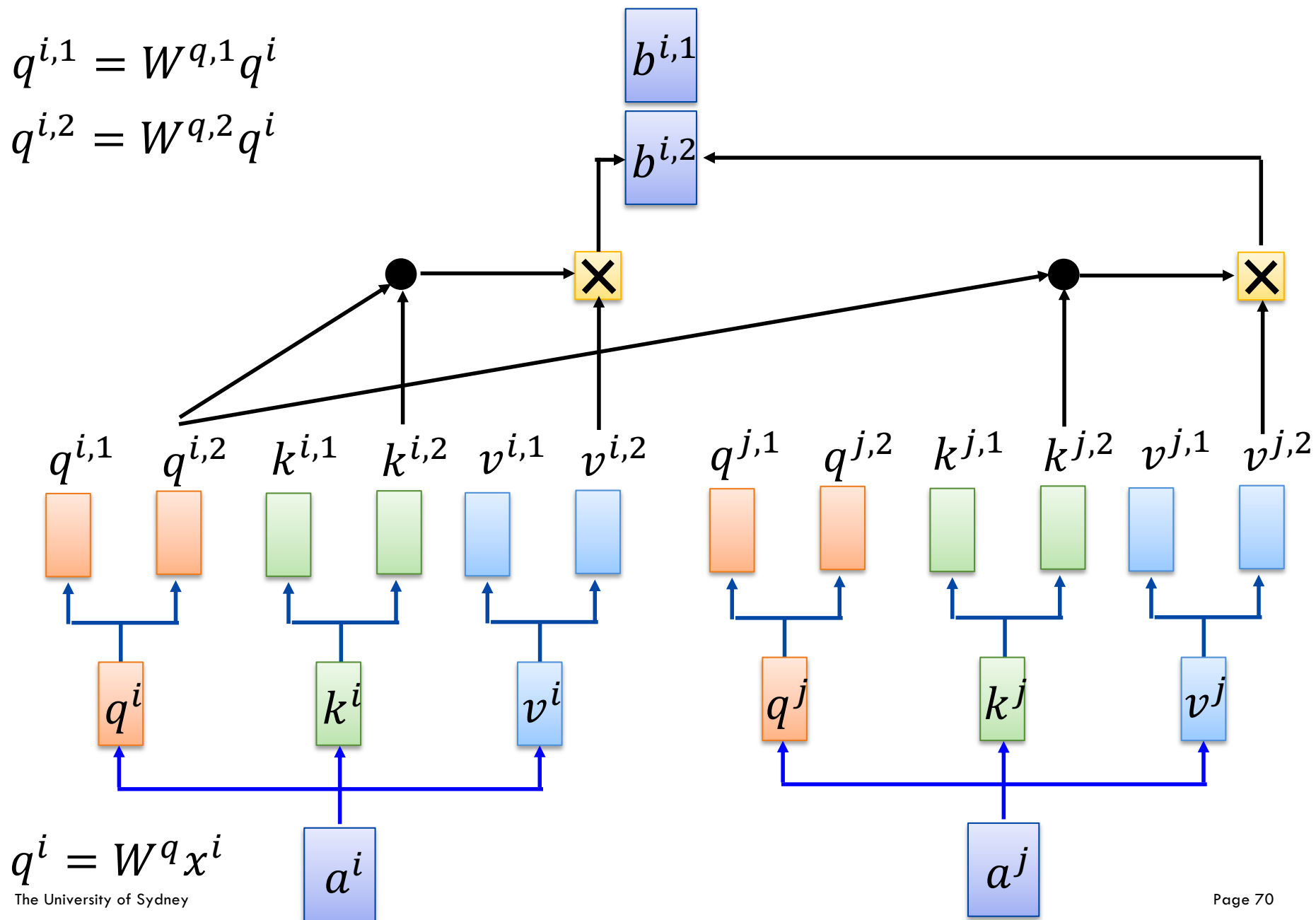


Multi-head Self-Attention

(2 heads as example)

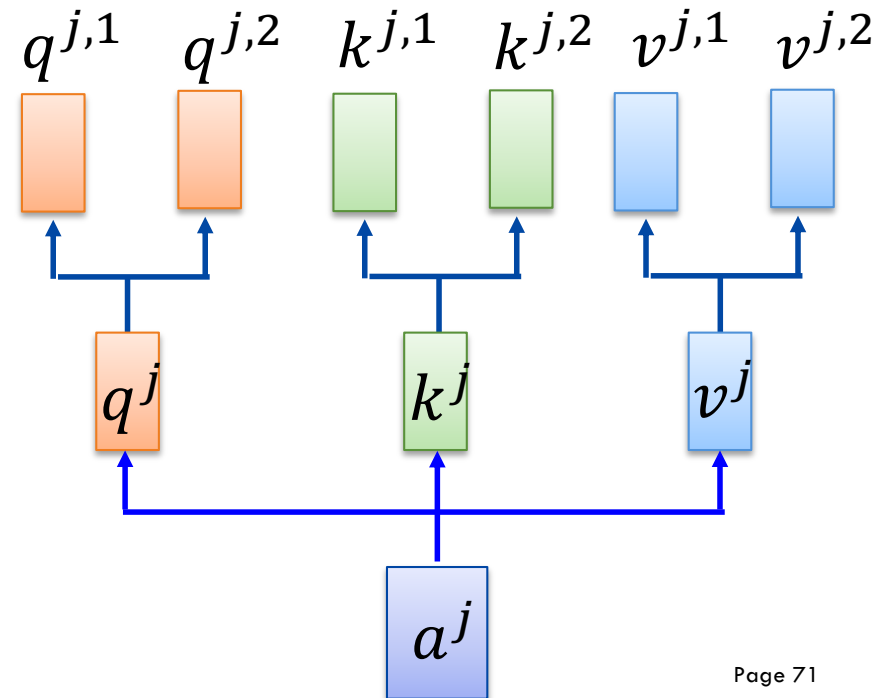
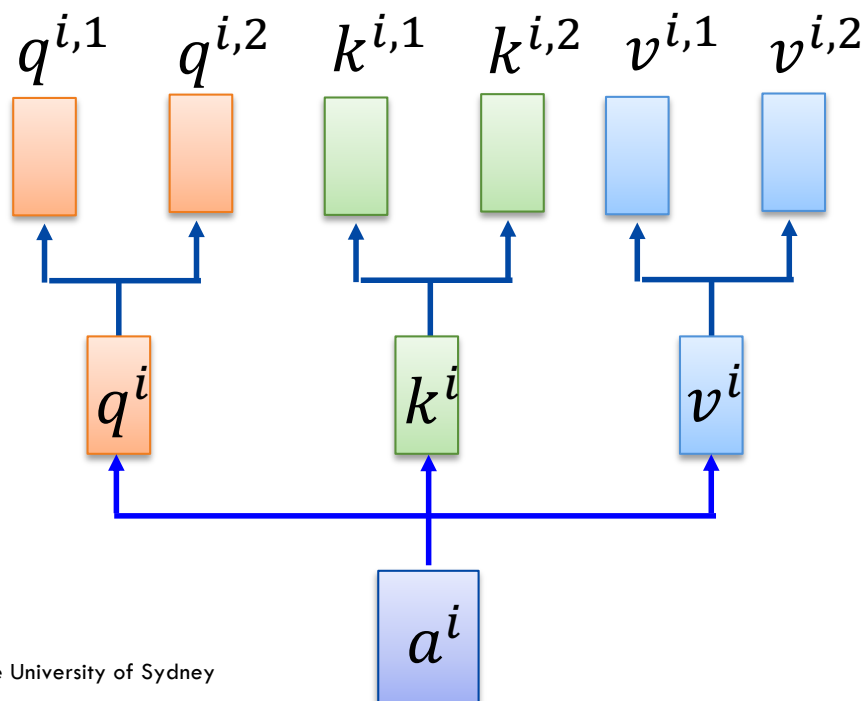
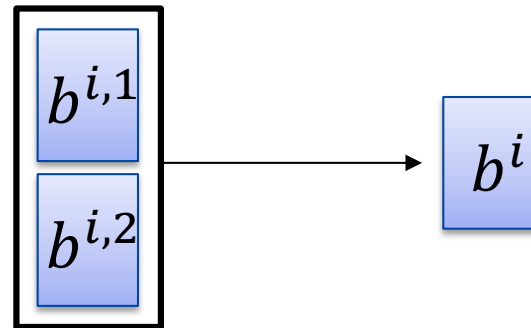
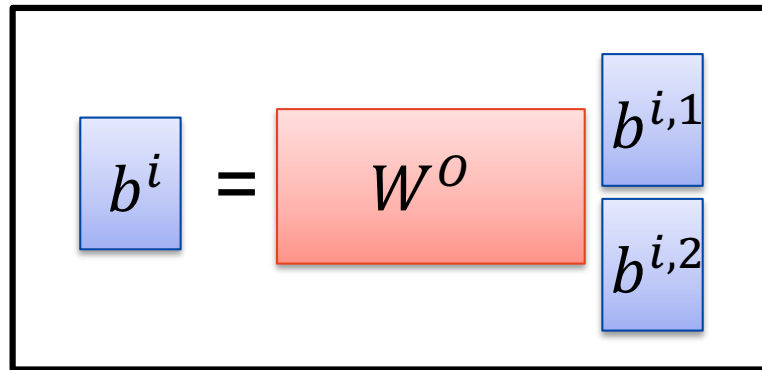
$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



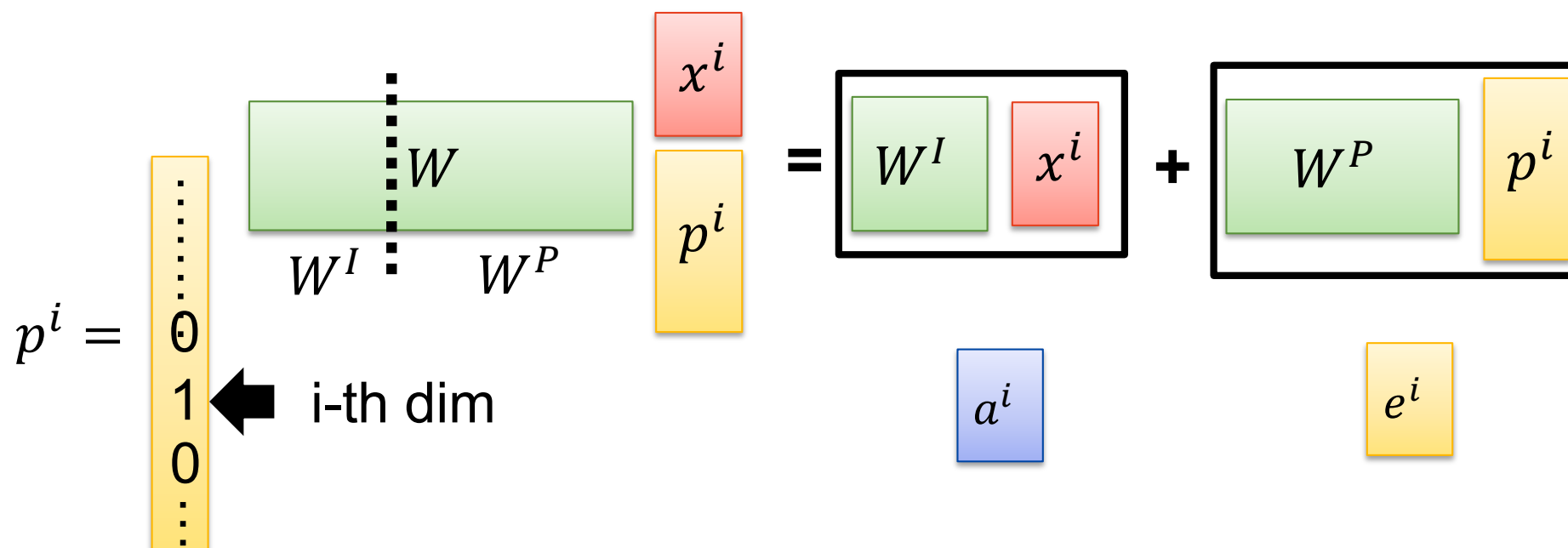
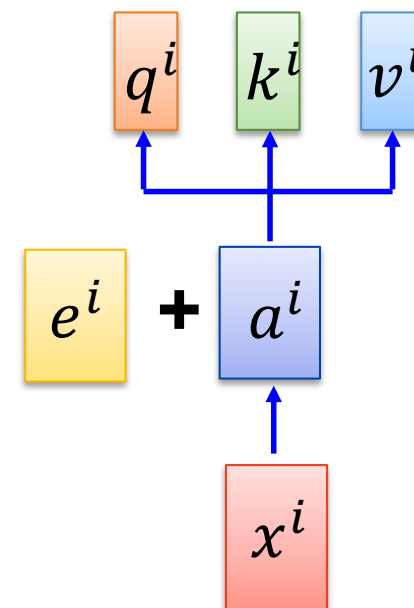
Multi-head Self-Attention

(2 heads as example)



Positional Encoding

- No position information in self-attention.
- Original paper: each position has a unique positional vector e^i (not learned from data)
- In other words: each x^i appends a one-hot vector p^i



Sinusoidal Positional Encoding

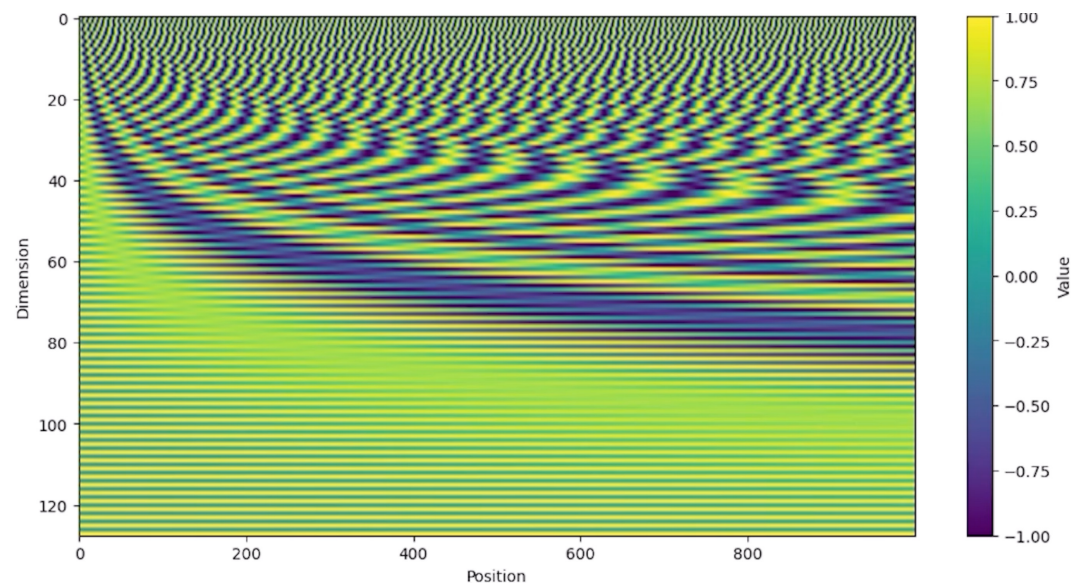
$$e^i(i, 2j) = \sin(i/10000^{2j/d})$$

$$e^i(i, 2j + 1) = \cos(i/10000^{2j/d})$$

i: token position in the
sequence

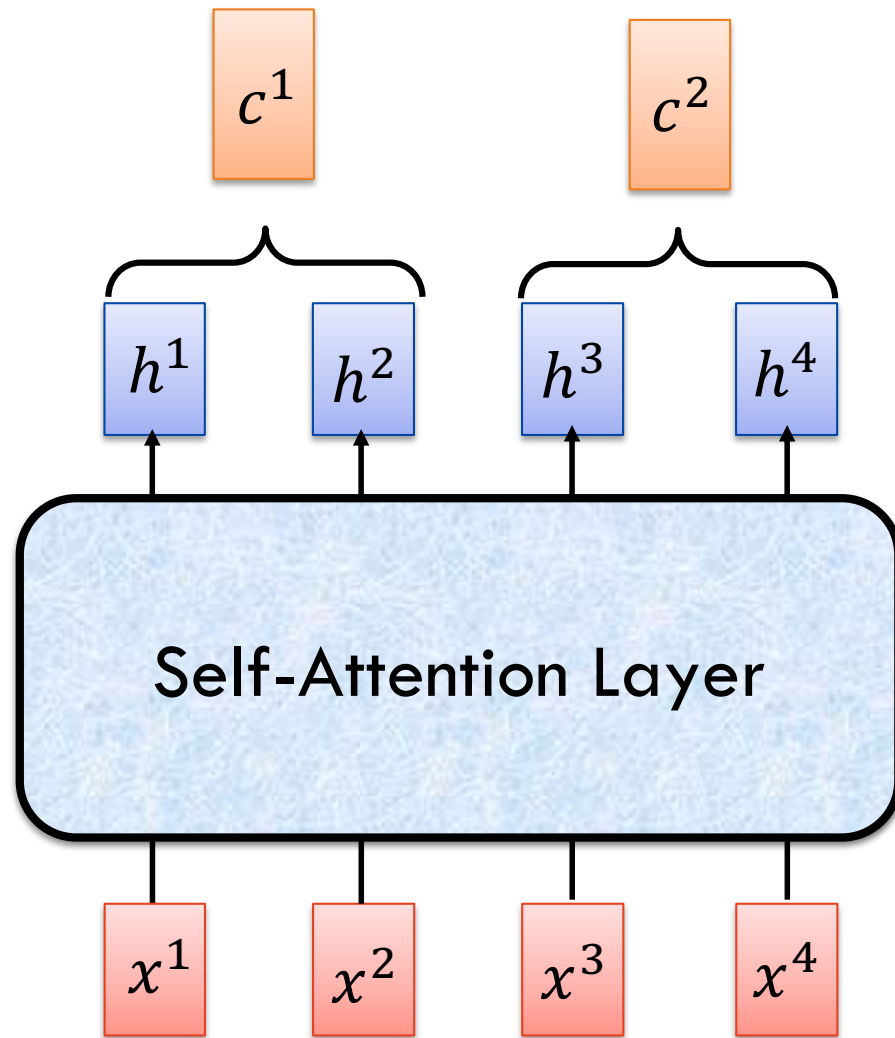
j: dimension index of the
positional encoding /
embedding

d: embedding dimension

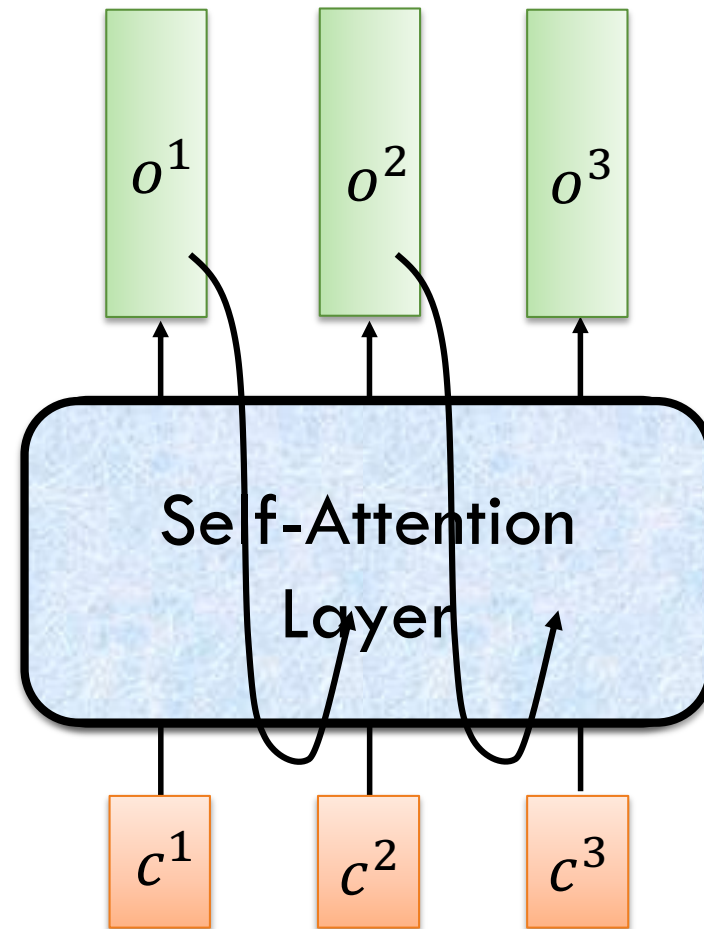


Lower dimensions vary faster, while higher dimensions vary more slowly, allowing the model to encode both local and global position patterns.

Seq2seq with Attention



Encoder

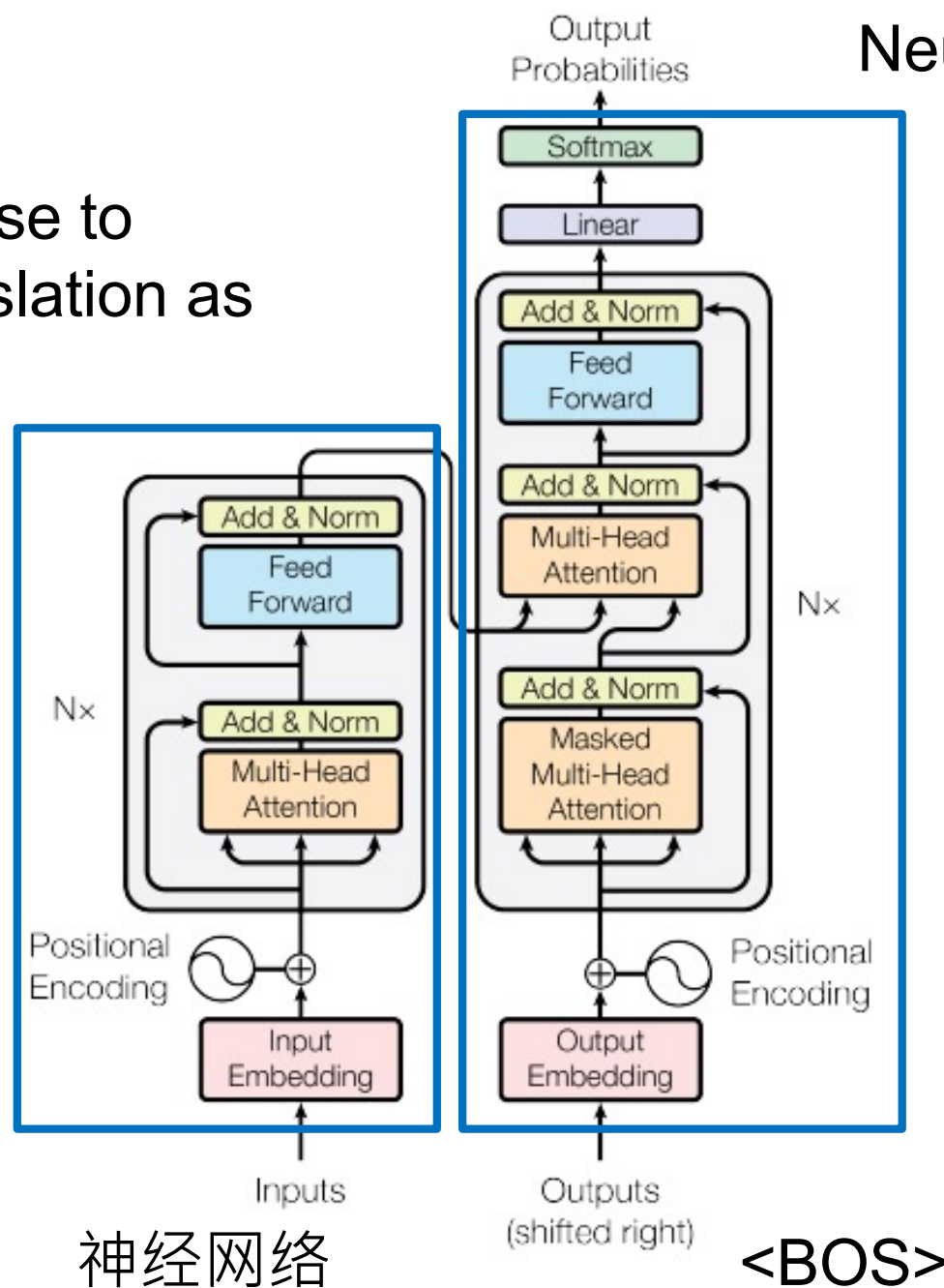


Decoder

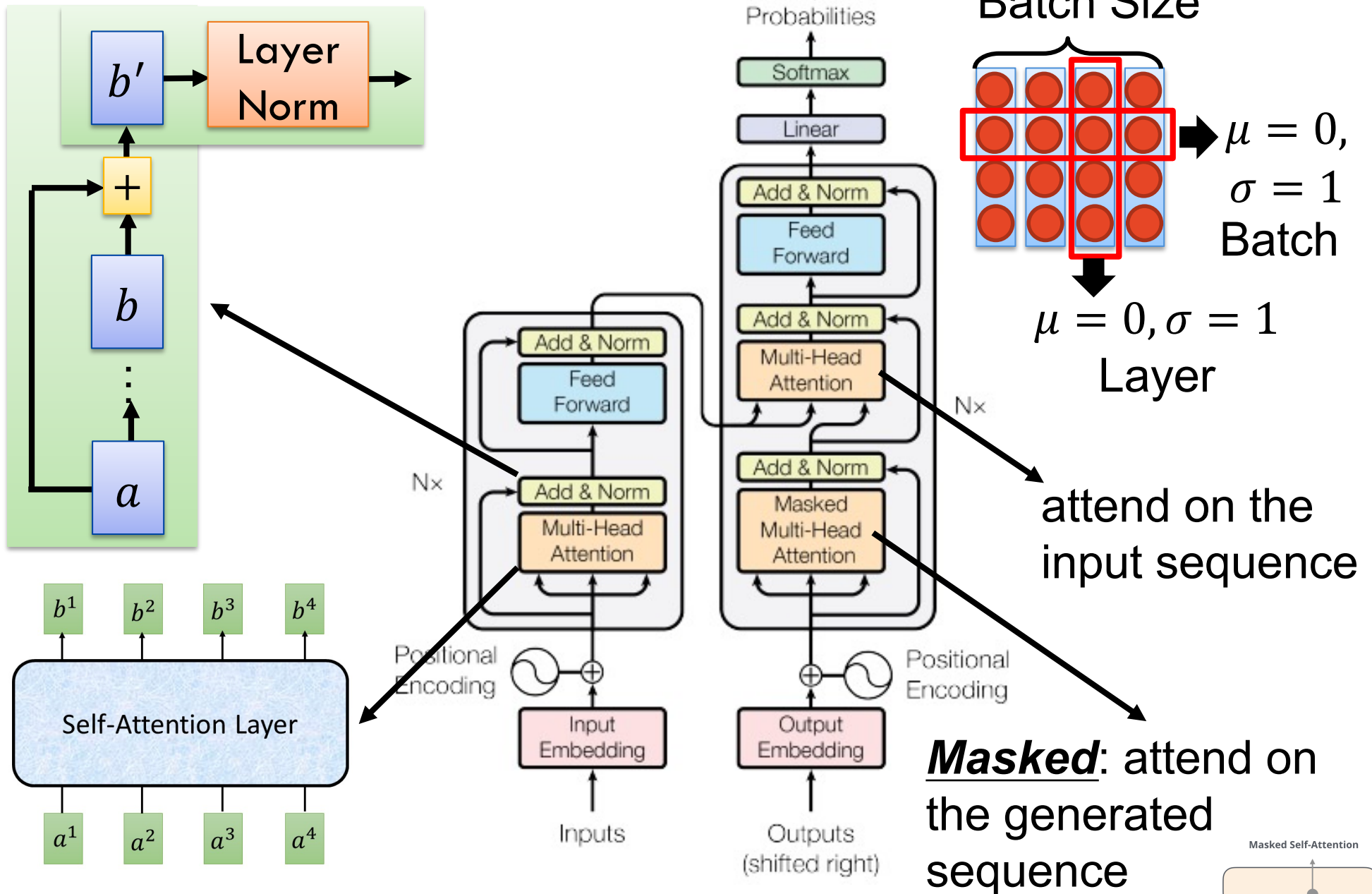
Transformer

Using Chinese to English translation as example

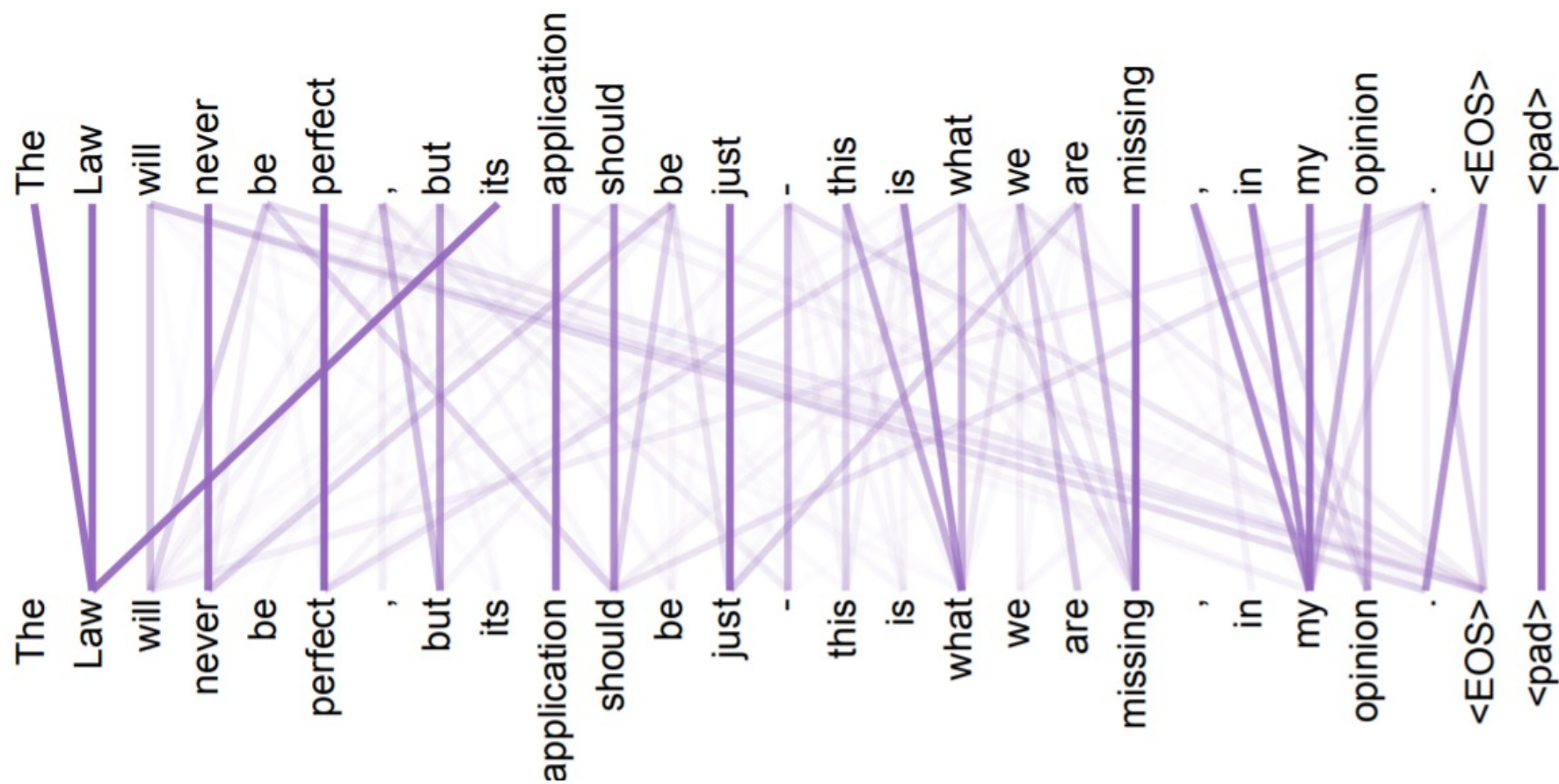
Encoder



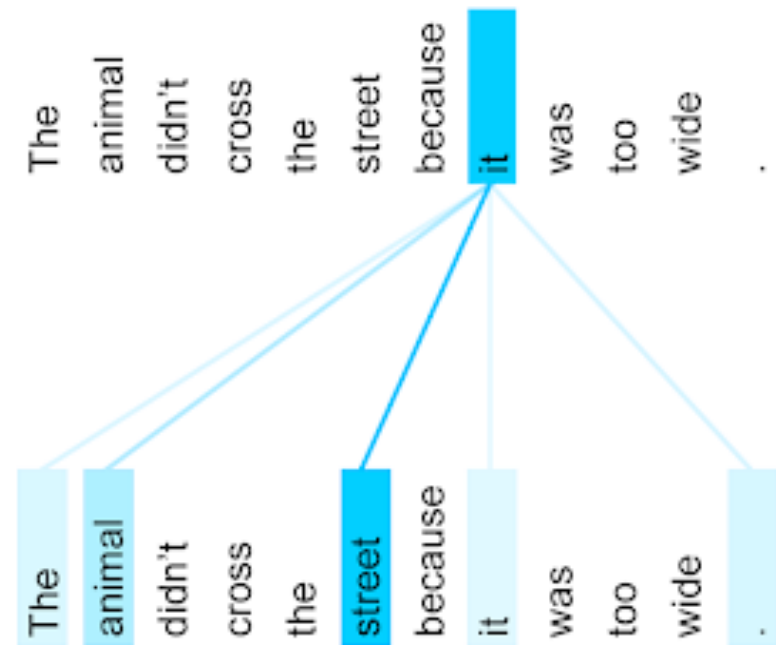
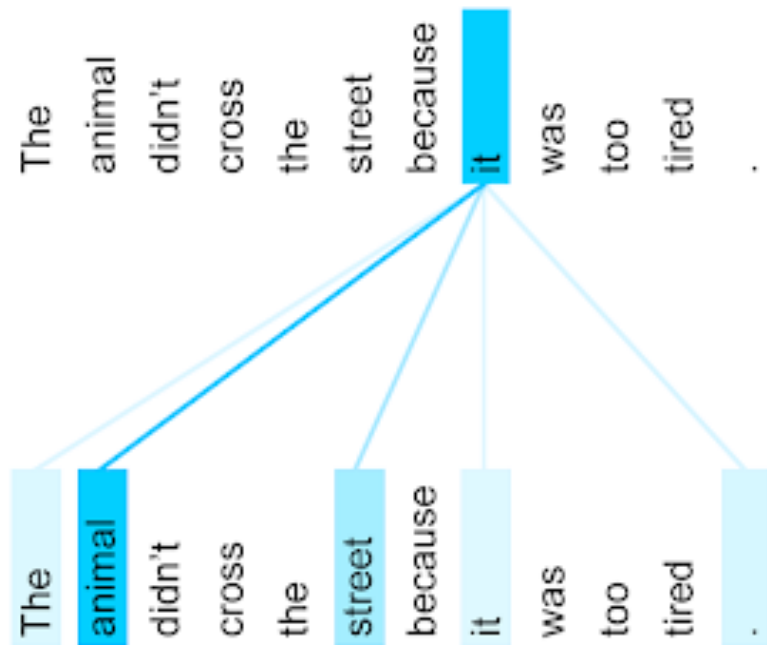
Transformer



Attention Visualization



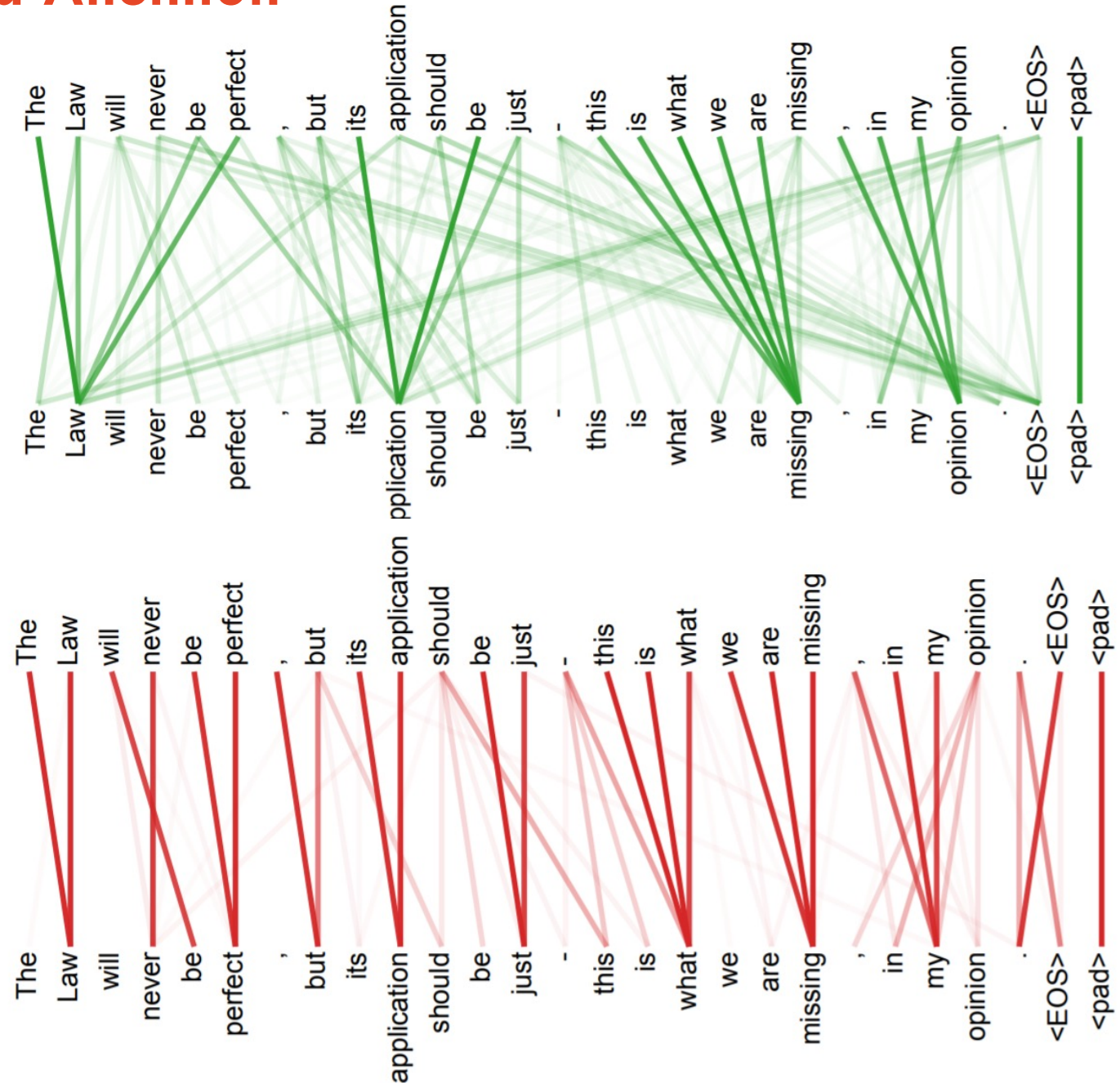
Attention Visualization



The encoder self-attention distribution for the word “it” from the 5th to the 6th layer of a Transformer trained on English to French translation (one of eight attention heads).

<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

Multi-head Attention



Thank you !